Tecnológico Nacional de México Instituto Tecnológico de Tijuana

Materia: Lenguajes y automatas II

Grupo: SC7C

Carrera: Sistemas Computacionales

Periodo: Agosto - Diciembre 2026

Docente: Luis Alfonso Gaxiola Vega

Alumnos:
Gonzalez Cristobal Omar
García Luna Ricardo Daniel

Unidad: 4

Actividad: UNIDAD 4. PROYECTO FINAL INTEGRADOR

TÍTULO DEL PROYECTO A REALIZAR: “CARDIO GUARD: INTÉRPRETE PARA LA VALIDACIÓN Y CLASIFICACIÓN DE LECTURAS DE PRESIÓN ARTERIAL”

Fecha: 9/11/2026

Cardio Guard — Documentación Final

1. Portada
2. Índice
3. Introducción y antecedentes
    - 3.1 Introducción
    - 3.2 Antecedentes
4. Objetivos
    - 4.1 Objetivo general
    - 4.2 Objetivos específicos
5. Marco teórico: problemática y solución
    - 5.1 Problemática
    - 5.2 Fundamentos de lenguajes y autómatas
    - 5.3 Fundamentos clínicos
    - 5.4 Tecnologías utilizadas
    - 5.5 Solución propuesta
6. Elaboración: pasos a seguir
    - 6.1 Arquitectura y flujo de una instrucción
    - 6.2 El lenguaje de Cardio Guard
    - 6.3 Análisis léxico (tokens, lexemas y reglas)
    - 6.4 Análisis sintáctico (gramática)
    - 6.5 Análisis semántico
    - 6.6 Clasificación clínica y comparación
    - 6.7 Autómata finito determinista (árbol de decisiones)
    - 6.8 Almacenamiento y bitácora
    - 6.9 Pasos seguidos en el desarrollo
7. Pantallas e interfaz de usuario
    - 7.1 Estructura general de la pantalla
    - 7.2 Pestañas de captura
    - 7.3 Tarjeta de resultado y estados
    - 7.4 Consulta y listado de pacientes
    - 7.5 Bitácora de la sesión
    - 7.6 Ayuda integrada y pantalla Acerca de
    - 7.7 Tema claro y oscuro, y vista móvil
8. Código generado en el lenguaje realizado
    - 8.1 Analizador léxico (lexer.js)
    - 8.2 Analizador sintáctico (parser.js)
    - 8.3 Autómata finito determinista (afd.js)
    - 8.4 Análisis semántico de la lectura (validarLectura.js)
    - 8.5 Clasificación NOM-030 y comparación (clasificacion.js)
    - 8.6 Intérprete (interpreter.js)
    - 8.7 Salida del análisis
9. Manuales

- Manual de usuario
  - 9.1 Antes de empezar
  - 9.2 Registrar un paciente
  - 9.3 Registrar una lectura
  - 9.4 Alertas y datos sospechosos
  - 9.5 Errores y rechazos
  - 9.6 Consultar un paciente
  - 9.7 Listar los pacientes
  - 9.8 Revisar la bitácora de la sesión
  - 9.9 Ayuda y uso en teléfono
  - 9.10 Mensajes del sistema y solución de problemas
- Manual de administrador
  - A.1 Requisitos
  - A.2 Descargar e instalar el proyecto
  - A.3 Configurar el archivo .env
  - A.4 Preparar SQL Server (solo modo SQL)
  - A.5 Arrancar y detener el servidor
  - A.6 Archivos que genera el sistema
  - A.7 Revisar los datos en SQL Server
  - A.8 Pruebas del sistema
  - A.9 Interfaz de programación (API)
  - A.10 Respaldo y solución de problemas

10. Conclusiones
11. Recomendaciones
12. Bibliografía
13. Anexos

3. Introducción y antecedentes

3.1 Introducción

Cardio Guard es un intérprete que recibe instrucciones de texto sobre pacientes de un hospital y las valida igual que un compilador valida un programa: primero revisa que cada símbolo sea válido (análisis léxico), después que la instrucción tenga la estructura correcta (análisis sintáctico) y por último que los datos tengan sentido (análisis semántico). Un autómata finito determinista (AFD) controla ese recorrido y decide si cada instrucción se acepta o se rechaza.

El lenguaje tiene cuatro instrucciones: registrar a un paciente, capturar una lectura de presión arterial y frecuencia cardíaca, consultar a un paciente y listar a todos. Cuando una lectura es aceptada, el sistema la clasifica según la norma mexicana de hipertensión, la compara con la revisión anterior del mismo paciente y, si los valores lo ameritan, muestra una alerta. Todo se usa desde una interfaz web y los datos se guardan en un archivo CSV o en una base de datos SQL Server.

Este documento reúne el análisis, el diseño, el código y los manuales del sistema terminado.

3.2 Antecedentes

El proyecto se desarrolló para la materia de Lenguajes y Autómatas II, cuyo propósito es aplicar las fases de análisis de un traductor a un problema concreto. Se eligió el monitoreo de signos vitales porque la presión arterial y la frecuencia cardíaca se registran con un formato corto y muy regular (por ejemplo, 118/76 mmHg y 75 lpm), lo que permite definir un lenguaje pequeño con reglas léxicas, sintácticas y semánticas claras.

Se optó por un **intérprete**, y no por un compilador o un traductor, porque el sistema no genera código ni traduce a otro lenguaje: ejecuta cada instrucción en cuanto la analiza.

Antes de programar, el equipo diseñó el lenguaje (tokens y gramática), el AFD, las reglas semánticas y el catálogo de errores, y lo presentó en un documento técnico y una exposición. En esa etapa se tomaron decisiones que se conservaron hasta el final: los pacientes se guardan en un repositorio aparte y no se repiten en cada lectura; registrar un paciente es una instrucción más del lenguaje, para que pase por las mismas validaciones; la fecha de cada revisión la asigna el sistema; y la comparación se hace solo contra la última revisión.

La primera versión clasificaba la presión con la guía estadounidense AHA/ACC 2017. Con esa guía, valores como 130/80 mmHg ya se consideran hipertensión, lo que resultaba más alarmante que el criterio que se usa en México. Por eso la clasificación se cambió a la de la norma mexicana. Después se agregaron las instrucciones de consulta y listado, se rediseñó la interfaz, se ajustaron las reglas de las lecturas y se añadió SQL Server con historial. Esas etapas se detallan en la sección 6.9.

4. Objetivos

4.1 Objetivo general

Desarrollar un intérprete que analice léxica, sintáctica y semánticamente instrucciones de registro y consulta de pacientes y de lecturas de presión arterial y frecuencia cardíaca, controlado por un autómata finito determinista, que acepte o rechace cada instrucción con un mensaje claro y clasifique las lecturas válidas según la norma mexicana de hipertensión.

4.2 Objetivos específicos

1. Definir un lenguaje de cuatro instrucciones (REGISTRAR, lectura, CONSULTAR y LISTAR) con sus tokens, lexemas y patrones.
2. Construir un analizador léxico que convierta cada línea en tokens y reporte la posición de cualquier símbolo no reconocido.
3. Construir un analizador sintáctico descendente recursivo basado en una gramática LL(1), que indique qué elemento falta o sobra.
4. Implementar las reglas semánticas del registro, la lectura y la consulta (rangos válidos, paciente existente, ID no duplicado, sistólica mayor que diastólica).
5. Diseñar e implementar un AFD con estados de aceptación y de rechazo que guíe el procesamiento de cada instrucción.
6. Clasificar la presión arterial y la frecuencia cardíaca, generar alertas en valores de riesgo y comparar cada lectura con la revisión anterior del paciente.
7. Guardar a los pacientes en un archivo CSV o en SQL Server, y registrar cada instrucción en una bitácora de auditoría.
8. Ofrecer una interfaz web que permita capturar las instrucciones y ver los resultados de forma clara, incluso en el teléfono.

5. Marco teórico: problemática y solución

5.1 Problemática

La hipertensión arterial es una de las enfermedades crónicas más comunes en México. Según los resultados de la Encuesta Nacional de Salud y Nutrición (Ensanut) 2021-2024, el 29.1 % de los adultos tiene hipertensión, y solo el 62.9 % de ellos ya había sido diagnosticado (Campos-Nonato et al., 2025). Es decir, más de un tercio de las personas con hipertensión no sabe que la tiene, por lo que medir y registrar la presión de forma constante es importante.

En un hospital, la presión y el pulso se miden muchas veces al día y se anotan a mano o se capturan en un sistema. En ese proceso aparecen tres problemas:

- **Errores de captura.** Un símbolo de más, una diagonal que falta o un número imposible (por ejemplo, una diastólica mayor que la sistólica) pueden guardarse sin que nadie lo note.
- **Clasificación inconsistente.** Si cada persona interpreta los valores por su cuenta, una misma lectura puede considerarse normal por unos y elevada por otros.
- **Cambios que pasan desapercibidos.** Sin comparar con la revisión anterior, es difícil ver si el paciente mejoró o empeoró, y un valor de emergencia puede no recibir atención a tiempo.

5.2 Fundamentos de lenguajes y autómatas

Traductores e intérpretes. Un compilador traduce un programa completo a otro lenguaje antes de ejecutarlo; un intérprete, en cambio, analiza y ejecuta las instrucciones una por una (Aho et al., 2008). Ambos comparten las mismas fases de análisis.

Análisis léxico. Es la primera fase. Lee la entrada carácter por carácter y la agrupa en *lexemas*, que son las secuencias de caracteres con significado. Cada lexema se clasifica en un *token* (una categoría, como número o identificador) según un *patrón*, que normalmente se escribe como expresión regular. Si un carácter no coincide con ningún patrón, se produce un error léxico (Aho et al., 2008).

Análisis sintáctico. Revisa que la secuencia de tokens siga la estructura del lenguaje, definida por una *gramática libre de contexto*: un conjunto de producciones que indican cómo se forma cada construcción a partir de símbolos terminales (tokens) y no terminales (reglas). Un analizador *descendente recursivo* tiene una función por cada no terminal; si puede elegir la producción correcta mirando solo el siguiente token, es *predictivo* y la gramática es LL(1). El resultado es un árbol sintáctico (Aho et al., 2008).

Análisis semántico. Comprueba que una construcción bien formada tenga sentido: tipos, rangos, existencia de los identificadores usados y otras reglas que la gramática no puede expresar (Aho et al., 2008).

Autómata finito determinista. Un AFD es una quíntupla $M = (Q, \Sigma, \delta, q0, F)$, donde Q es un conjunto finito de estados, Σ el alfabeto de entrada, δ la función de transición que a cada par (estado, símbolo) le asigna un único estado siguiente, q0 el estado inicial y F el conjunto de estados de aceptación. Una entrada se acepta si, al terminar de leerla, el autómata queda en un estado de F (Hopcroft et al., 2007). En Cardio Guard, los símbolos del alfabeto son los eventos que produce cada fase del análisis.

5.3 Fundamentos clínicos

Presión arterial. Es la fuerza que ejerce la sangre sobre las paredes de las arterias. Se expresa en milímetros de mercurio (mmHg) con dos valores: la *sistólica*, cuando el corazón se contrae, y la *diastólica*, cuando se relaja. Por eso la sistólica siempre es mayor. La diferencia entre ambas se llama presión de pulso.

Frecuencia cardíaca. Es el número de latidos por minuto (lpm). En un adulto en reposo se considera normal entre 60 y 100 lpm; por debajo se habla de bradicardia y por encima de taquicardia.

Norma mexicana. La clasificación que usa el sistema proviene del Proyecto de Norma Oficial Mexicana PROY-NOM-030-SSA2-2017, para la prevención, detección, diagnóstico, tratamiento y control de la hipertensión arterial sistémica, publicado como proyecto de modificación de la NOM-030-SSA2-2009 (Secretaría de Salud, 2010; 2017). Divide la presión en óptima, subóptima, limítrofe e hipertensión de grados 1, 2 y 3, y considera hipertensión a partir de 140/90 mmHg. El sistema agrega la categoría *crisis hipertensiva* (más de 180/120 mmHg), que no forma parte de la norma, para disparar la alerta de emergencia. Los umbrales exactos están en la sección 9.3.

5.4 Tecnologías utilizadas

Tecnología	Uso en el proyecto
JavaScript y Node.js	Lenguaje y entorno en que se escribió el intérprete
Express	Servidor web y API que reciben las instrucciones
HTML, CSS y JavaScript	Interfaz web
SQL Server Express	Base de datos opcional con historial de revisiones
mssql y dotenv	Conexión a SQL Server y lectura de la configuración
Git y GitHub	Control de versiones y trabajo en equipo

5.5 Solución propuesta

Cardio Guard aplica las fases de un traductor a la captura de signos vitales:

- Las lecturas se escriben en un lenguaje con formato fijo (`P001; 118/76, 75`), de modo que el **análisis léxico y sintáctico** detecta los errores de captura y señala dónde están.
- El **análisis semántico** rechaza los valores imposibles y marca como sospechosos los extremos, sin perderlos.
- La **clasificación automática** aplica siempre los mismos umbrales de la norma mexicana, y la **comparación** con la revisión anterior indica si el paciente mejoró, empeoró o sigue igual.
- Las **alertas** destacan en rojo las crisis hipertensivas y las frecuencias severas.
- El **AFD** hace que cada instrucción siga un camino definido y termine en un estado de aceptación o de rechazo, y la **bitácora** deja constancia de todo lo procesado.

Las alertas y la clasificación son una ayuda para el personal y no sustituyen la valoración de un médico.

6. Elaboración: pasos a seguir

Esta sección explica cómo se construyó el intérprete de Cardio Guard: qué hace cada etapa, qué reglas aplica y en qué orden se desarrolló. El código de cada parte se presenta en la sección 8.

6.1 Arquitectura y flujo de una instrucción

El sistema está escrito en Node.js. La interfaz web solo arma el texto de la instrucción y lo envía al servidor Express por `POST /api/procesar`; quien valida todo es el intérprete. Por cada línea recibida, `procesarLinea()` crea un autómata nuevo y pasa el texto por cuatro etapas: análisis léxico, análisis sintáctico, análisis semántico y clasificación clínica. Cada etapa entrega un evento al AFD, y el estado al que llega decide si la instrucción se acepta (y se guarda) o se rechaza. Toda instrucción, aceptada o no, queda en la bitácora de auditoría.

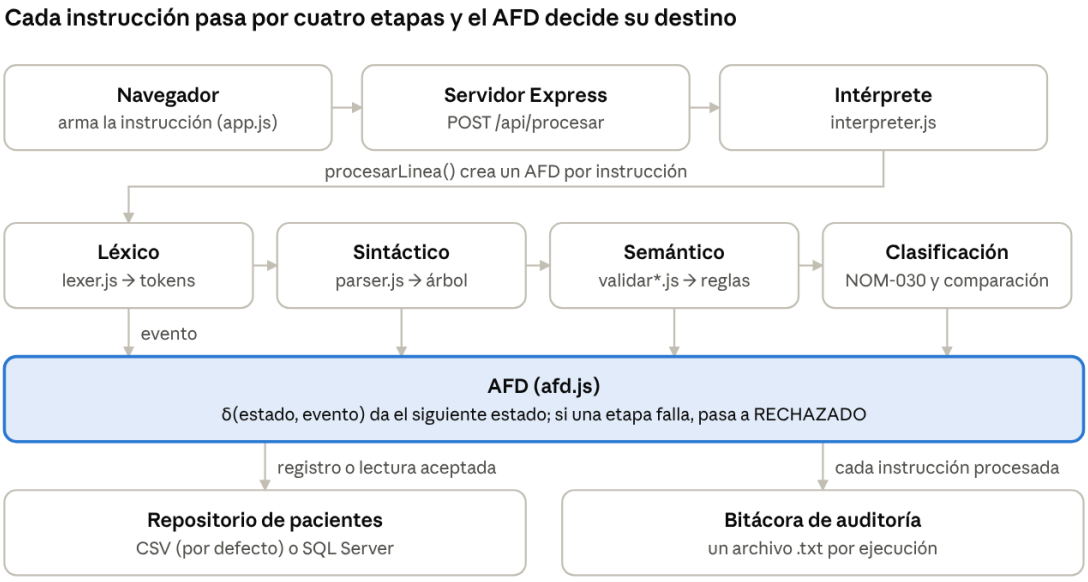


Figura 6.1 · Flujo de una instrucción por el intérprete

Carpeta / archivo	Responsabilidad
`public/` (index.html, app.js, style.css)	Interfaz web: pestañas de captura, resultado, ficha, tabla de pacientes y bitácora de sesión
`src/server.js`	Servidor Express y API (`/api/procesar`, `/api/estado`, `/api/pacientes`)
`src/interpreter.js`	Orquesta las etapas y mueve el AFD
`src/lexer/lexer.js`	Análisis léxico
`src/parser/parser.js`	Análisis sintáctico

Carpeta / archivo	Responsabilidad
`src/automaton/afd.js`	Autómata finito determinista
`src/semantics/`	Reglas semánticas y clasificación NOM-030
`src/storage/`	Repositorio CSV o SQL Server y bitácora
`sql/`	Scripts para crear la base, el usuario y migrar el historial

6.2 El lenguaje de Cardio Guard

El lenguaje tiene cuatro instrucciones, una por línea:

Instrucción	Forma	Ejemplo
Registro de paciente	`REGISTRAR id, edad, sexo, sangre`	`REGISTRAR P001, 45, M, O+`
Lectura	`id; sistólica/diastólica, frecuencia`	`P001; 118/76, 75`
Consulta	`CONSULTAR id`	`CONSULTAR P001`
Listado	`LISTAR`	`LISTAR`

La fecha de cada revisión no se escribe en la instrucción: la asigna el sistema con la hora local de la computadora.

6.3 Análisis léxico (tokens, lexemas y reglas)

El analizador léxico recorre la línea de izquierda a derecha, ignora los espacios y, en cada posición, prueba las reglas en un orden fijo. La primera expresión regular que coincide define el token. El orden importa: las palabras reservadas van antes que `LETRA` y `SANGRE` va antes que `LETRA`, para que una regla general no se quede con un texto que pertenece a una más específica.

#	Token	Patrón (regex)	Lexemas de ejemplo	Descripción
1	REGISTRAR	`^REGISTRAR\b`	REGISTRAR	Palabra reservada
2	CONSULTAR	`^CONSULTAR\b`	CONSULTAR	Palabra reservada
3	LISTAR	`^LISTAR\b`	LISTAR	Palabra reservada
4	ID	`^P\d+`	P001, P12	Identificador de paciente
5	SANGRE	`^[A-Za-z]+[+-]`	O+, AB-	Tipo de sangre (letras seguidas de signo)
6	NUM	`^\d+`	45, 118, 75	Número entero sin signo

#	Token	Patrón (regex)	Lexemas de ejemplo	Descripción
7	LETRA	`^[A-Za-z]+`	M, F	Letras sueltas (el parser decide si es el sexo)
8	SLASH	`^/`	/	Separa sistólica y diastólica
9	COMA	`^,`	,	Separador de campos
10	PUNTOYCOMA	`^;`	;	Separa el ID de la lectura
11	GUION	`^-`	-	Token propio para detectar negativos como `120/-80`
12	EOF	(fin de línea)	—	Marca el final de la instrucción

Cada token se guarda con su tipo, su lexema (el texto exacto) y su posición en la línea. Así queda la línea `P001; 118/76, 75`:

Lexema	Token	Posición
P001	ID	0
;	PUNTOYCOMA	4
118	NUM	6
/	SLASH	9
76	NUM	10
,	COMA	12
75	NUM	14
—	EOF	16

Si en una posición no coincide ninguna regla, el analizador lanza un `ErrorLexico` con el símbolo y su posición. Por ejemplo, en `P001; 120#80, 75` el símbolo `#` en la posición 9 produce *Error Léxico en la línea 1: Símbolo no reconocido*, y la interfaz marca con `^` el lugar exacto.

6.4 Análisis sintáctico (gramática)

La gramática del lenguaje, en forma de producciones, es:

```
S   -> REG | LEC | CON | LIS
REG -> registrar id , num , sexo , sangre
LEC -> id ; P , F
P   -> num / num
F   -> num
CON -> consultar id
LIS -> listar
```

El analizador es **descendente recursivo y predictivo**: hay una función por cada regla (`parsearREG`, `parsearLEC`, `parsearP`, `parsearF`, `parsearCON`, `parsearLIS`) y la regla de `S` se elige mirando solo el primer token, sin retroceder. Como cada alternativa empieza con un token distinto (REGISTRAR, ID, CONSULTAR o LISTAR), la gramática es LL(1). Al terminar cada instrucción se exige `EOF`, para que no sobre texto al final.

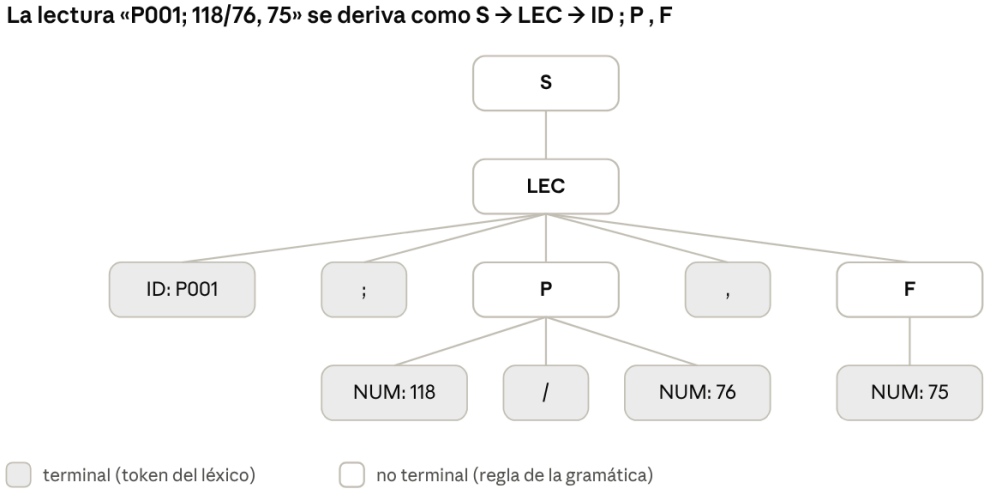


Figura 6.2 · Árbol sintáctico de una lectura

El resultado es un árbol simplificado en forma de objeto, por ejemplo `{"tipo":"LEC","id":"P001","sistolica":118,"diastolica":76,"frecuencia":75}`, que es lo que reciben las etapas siguientes.

Cuando un token no es el que espera la regla, se lanza un `ErrorSintactico` con un detalle específico y la posición del token:

Situación	Detalle que se muestra
La línea no empieza con una instrucción válida	se esperaba REGISTRAR, CONSULTAR, LISTAR o el identificador de un paciente
Falta el `;` después del ID	se esperaba ";"
Falta un número de la presión	se esperaba la presión sistólica / la presión diastólica
Falta la diagonal	se esperaba "/"
Diastólica negativa (`120/-80`)	se esperaba un número positivo después de la diagonal
Falta la frecuencia	se esperaba la frecuencia cardíaca
Falta un campo del registro	se esperaba el identificador del paciente / la edad / el sexo / el tipo de sangre

Situación	Detalle que se muestra
Falta una coma	se esperaba ","
Texto de más al final	sobra texto después de la instrucción

En la interfaz todos se reportan como *Error de Sintaxis en la línea N: Estructura de tokens inválida*, seguido del detalle y el marcador `^`.

6.5 Análisis semántico

Una instrucción bien escrita todavía puede no tener sentido. El análisis semántico revisa el significado de los datos:

Instrucción	Regla	Si no se cumple
Registro	El ID tiene como máximo 20 caracteres (igual que la columna `VARCHAR(20)`)	Rechazo
Registro	El ID no está registrado ya	Rechazo
Registro	Edad entre 0 y 120	Rechazo
Registro	Sexo M o F	Rechazo
Registro	Sangre: A+, A-, B+, B-, AB+, AB-, O+ u O-	Rechazo
Lectura	El paciente existe	Rechazo
Lectura	Sistólica mayor que diastólica (120/120 no es válido)	Rechazo
Lectura	Ningún valor en cero o negativo	Rechazo
Lectura	Frecuencia entre 30 y 220 lpm	Rechazo
Consulta	El paciente existe	Rechazo

Además, una lectura válida puede marcarse como **dato sospechoso** (advertencia, no rechazo) cuando la presión es extrema (sistólica > 250 o diastólica > 150), la presión de pulso es muy baja (sistólica − diastólica < 10) o la frecuencia está en un extremo (< 40 o > 200 lpm).

6.6 Clasificación clínica y comparación

Las lecturas aceptadas se clasifican con la NOM-030-SSA2-2017. Cada categoría tiene un nivel numérico, de 0 (óptima) a 6 (crisis hipertensiva); las categorías se evalúan de la más grave a la menos grave y se toma la primera que coincide, de modo que si la sistólica y la diastólica caen en categorías distintas, se usa la más alta. La frecuencia cardíaca se clasifica como bradicardia (< 60), normal (60–100) o taquicardia (> 100). Los umbrales completos están en la sección 9.3.

Se genera una **alerta** cuando hay crisis hipertensiva (sistólica > 180 o diastólica > 120) o frecuencia severa (< 40 bradicardia severa, > 150 taquicardia severa). Con una alerta no se muestra recomendación de texto, para no contradecir la urgencia.

Después se compara contra la última revisión del paciente por niveles. La regla es conservadora: si la presión o la frecuencia empeoraron, el resultado es NEGATIVO aunque la otra haya mejorado; si nada empeoró y algo mejoró, es POSITIVO; si no cambió nada, SIN_CAMBIO; y si no hay revisión anterior, PRIMERA_REVISION.

6.7 Autómata finito determinista (árbol de decisiones)

El AFD controla el camino de cada instrucción. Se define como la quíntupla M = (Q, Σ, δ, q0, F):

- **Q** (estados): INICIO, VALIDANDO_SEMANTICA_REGISTRO, BUSCANDO_PACIENTE, VALIDANDO_SEMANTICA_LECTURA, CLASIFICANDO, BUSCANDO_PACIENTE_CONSULTA, LISTANDO, ACEPTADO, ACEPTADO_CON_ALERTA, ACEPTADO_CON_ADVERTENCIA, ACEPTADO_CON_ALERTA_Y_ADVERTENCIA y RECHAZADO.
- **Σ** (eventos): los resultados de cada etapa, como LEXICO_SINTACTICO_OK_REG, PACIENTE_ENCONTRADO o SEMANTICA_LECTURA_ERROR.
- **δ** (transición): una tabla con claves `ESTADO:EVENTO`.
- **q0**: INICIO.
- **F** (aceptación): ACEPTADO, ACEPTADO_CON_ALERTA, ACEPTADO_CON_ADVERTENCIA y ACEPTADO_CON_ALERTA_Y_ADVERTENCIA. RECHAZADO también es un estado final, pero queda fuera de F a propósito.

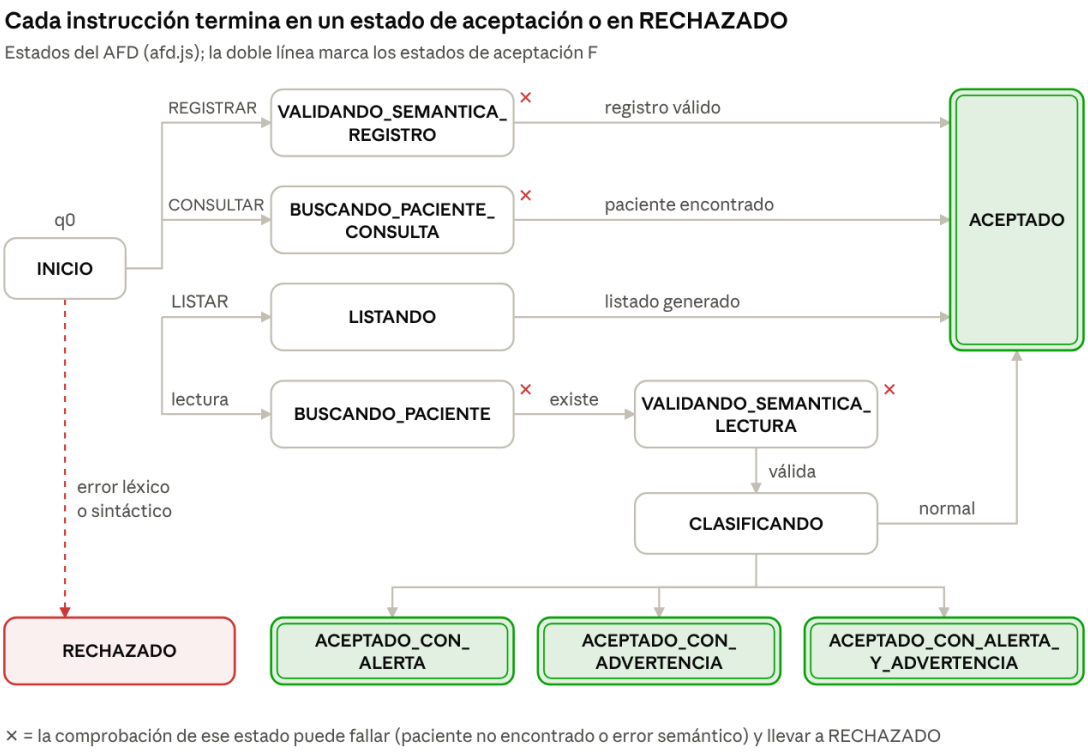


Figura 6.3 · Diagrama de estados del AFD (árbol de decisiones)

Estado actual	Evento	Estado siguiente
INICIO	LEXICO_SINTACTICO_ERROR	RECHAZADO
INICIO	LEXICO_SINTACTICO_OK_REG	VALIDANDO_SEMANTICA_REGISTRO
INICIO	LEXICO_SINTACTICO_OK_LEC	BUSCANDO_PACIENTE
INICIO	LEXICO_SINTACTICO_OK_CON	BUSCANDO_PACIENTE_CONSULTA
INICIO	LEXICO_SINTACTICO_OK_LIS	LISTANDO
VALIDANDO_SEMANTICA_REGISTRO	SEMANTICA_REGISTRO_ERROR	RECHAZADO
VALIDANDO_SEMANTICA_REGISTRO	SEMANTICA_REGISTRO_OK	ACEPTADO
BUSCANDO_PACIENTE_CONSULTA	CONSULTA_PACIENTE_NO_ENCONTRADO	RECHAZADO
BUSCANDO_PACIENTE_CONSULTA	CONSULTA_PACIENTE_ENCONTRADO	ACEPTADO
LISTANDO	LISTADO_OK	ACEPTADO
BUSCANDO_PACIENTE	PACIENTE_NO_ENCONTRADO	RECHAZADO
BUSCANDO_PACIENTE	PACIENTE_ENCONTRADO	VALIDANDO_SEMANTICA_LECTURA
VALIDANDO_SEMANTICA_LECTURA	SEMANTICA_LECTURA_ERROR	RECHAZADO
VALIDANDO_SEMANTICA_LECTURA	SEMANTICA_LECTURA_OK	CLASIFICANDO
CLASIFICANDO	CLASIFICACION_NORMAL	ACEPTADO
CLASIFICANDO	CLASIFICACION_ALERTA	ACEPTADO_CON_ALERTA
CLASIFICANDO	CLASIFICACION_ADVERTENCIA	ACEPTADO_CON_ADVERTENCIA
CLASIFICANDO	CLASIFICACION_ALERTA_Y_ADVERTENCIA	ACEPTADO_CON_ALERTA_Y_ADVERTENCIA

Si el intérprete intentara un evento que la tabla no contempla para el estado actual, el AFD lanza `TransicionInvalidaError`. Ese error señala una falla del programa, no de los datos del usuario.

6.8 Almacenamiento y bitácora

El repositorio se elige con la variable `REPOSITORIO` del archivo `.env`: `csv` (valor por defecto) o `sql`. Con CSV, cada lectura aceptada reemplaza la revisión anterior del paciente. Con SQL Server se usan las tablas `pacientes` y `revisiones`, `CONSULTAR` muestra las 10 revisiones más recientes y cada lectura actualiza al paciente y agrega su revisión en una sola transacción, de modo que se guarda todo o nada. La aplicación se conecta con el usuario `cardio_guard_app`, que solo tiene permisos de lectura y escritura de datos. La configuración está en el manual de administrador.

Cada ejecución del servidor crea su propia bitácora (`bitacora_AAAA-MM-DD_HH-MM.txt`), que nunca sobrescribe una anterior. En ella se registra cada instrucción con su resultado, y también se imprime en la consola.

6.9 Pasos seguidos en el desarrollo

1. **Diseño y documentación.** Antes de programar se definieron los tokens, la gramática, el AFD, las reglas semánticas y el catálogo de errores. Se decidió que los pacientes vivieran en un repositorio aparte, que registrar fuera una instrucción del lenguaje y que la fecha la pusiera el sistema.
2. **Primera versión funcional.** Se programó la cadena completa (lexer, parser, AFD, semántica, almacenamiento en CSV y servidor Express) con la interfaz web como entrada.
3. **Ampliación del lenguaje.** Se agregaron `CONSULTAR` y `LISTAR` con sus propios estados, el marcador `^` en los errores, y se cambió la clasificación de la guía AHA/ACC 2017 a la NOM-030-SSA2-2017.
4. **Rediseño de la interfaz.** Captura por campos en cinco pestañas, ficha de paciente, tabla, bitácora con filtros, modo oscuro y diseño adaptable.
5. **Reglas de lectura y alertas.** Se rechazó la sistólica igual a la diastólica, se definieron las causas de dato sospechoso, se agregó la alerta por frecuencia severa y se quitó la recomendación cuando hay alerta.
6. **SQL Server e historial.** Se agregó SQL Server como opción, con historial de revisiones y transacciones.
7. **Pulido.** Se agregó `GET /api/pacientes` para sugerir IDs al cargar la página, animaciones que respetan el movimiento reducido y precarga de fuentes.

7. Pantallas e interfaz de usuario

La interfaz de Cardio Guard es una sola página web (http://localhost:3000) donde el usuario escribe o arma una instrucción, la envía al intérprete y ve al instante su estado, su clasificación clínica y su recomendación. Esta sección describe cada parte de la pantalla; el uso paso a paso de cada función está en el Manual de usuario (sección 9).

La interfaz no valida ni clasifica nada por sí misma: solo arma el texto de la instrucción y muestra la respuesta del servidor. Todo el análisis (léxico, sintáctico, semántico y el autómata) ocurre en el intérprete.

7.1 Estructura general de la pantalla

La página es una columna centrada que se lee de arriba hacia abajo.

Zona	Qué contiene
Cabecera	Nombre del sistema, descripción y enlaces **Inicio** y **Acerca de**.
Pestañas de captura	**Registro, Lectura, Consultar, Listar y Directa**: una por cada forma de enviar una instrucción.
Cuadro «Se enviará» y botón **Procesar**	Muestra el texto exacto que recibirá el intérprete antes de enviarlo.
«¿Cómo escribo una instrucción?»	Ayuda plegable con el formato de cada instrucción y el significado de cada estado.
Resumen de la ejecución	Contadores de instrucciones **procesadas, aceptadas, con alerta y rechazadas**, los rechazos por tipo, la fecha de procesamiento y el archivo de bitácora en uso.
Último resultado	Tarjeta con el resultado de la instrucción más reciente.
Bitácora de la sesión	Historial de las instrucciones enviadas desde que se abrió la página, con filtros por estado.
Pie	Leyenda fija «Resultados de carácter orientativo, no es diagnóstico médico» y enlace a Acerca de.

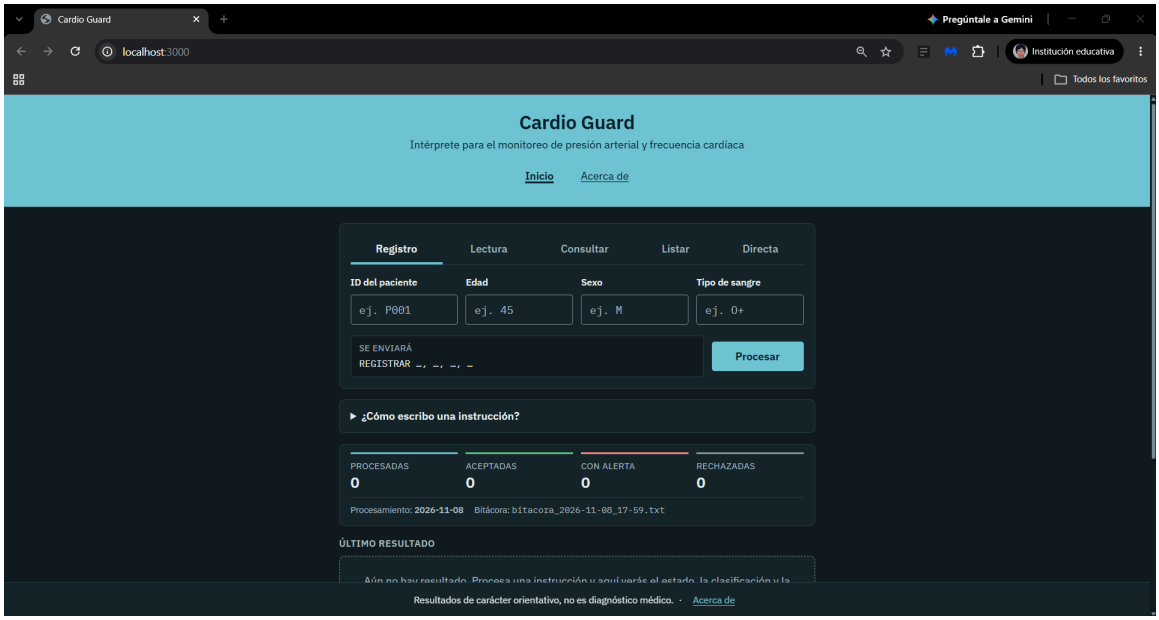


Figura 7.1. Pantalla inicial, sin instrucciones procesadas (captura 02).

7.2 Pestañas de captura

Cada pestaña arma una de las cuatro instrucciones del lenguaje a partir de campos sencillos; la pestaña **Directa** permite escribirla tal cual.

Pestaña	Campos	Instrucción que arma
Registro	ID del paciente, edad, sexo, tipo de sangre	`REGISTRAR P001, 45, M, O+`
Lectura	ID del paciente, presión sistólica, presión diastólica, frecuencia cardíaca	`P001; 118/76, 75`
Consultar	ID del paciente	`CONSULTAR P001`
Listar	Ninguno	`LISTAR`
Directa	Un campo de texto libre y cuatro ejemplos que se pueden pulsar para rellenarlo	Cualquiera de las anteriores

El cuadro «Se enviará» se actualiza mientras se escribe, así el usuario ve la instrucción formal que genera cada formulario. En **Lectura** y **Consultar**, el campo del ID ofrece una lista desplegable con los pacientes ya registrados, que se carga al abrir la página.

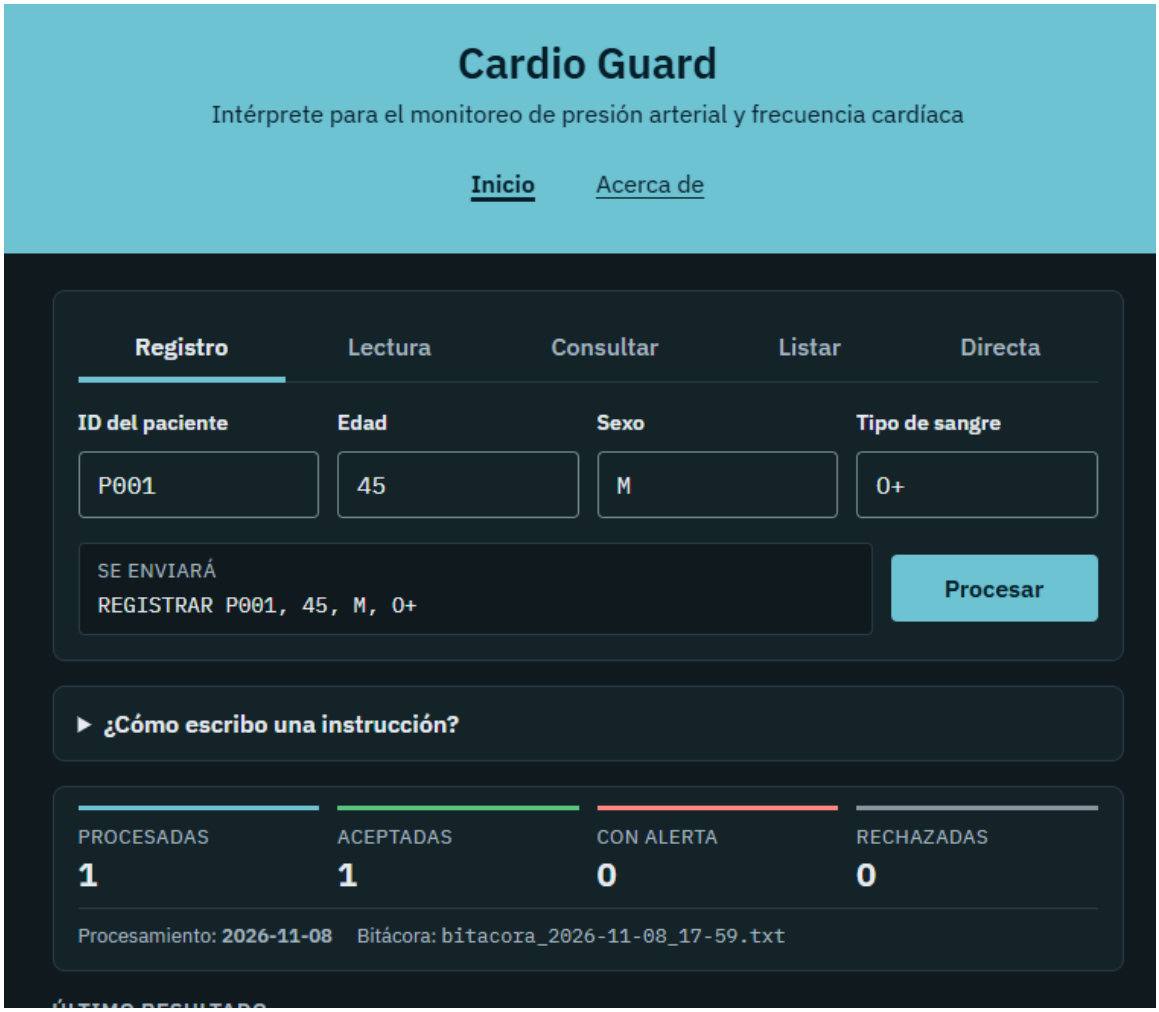


Figura 7.2. Pestaña Registro con el cuadro «Se enviará» (captura 04).

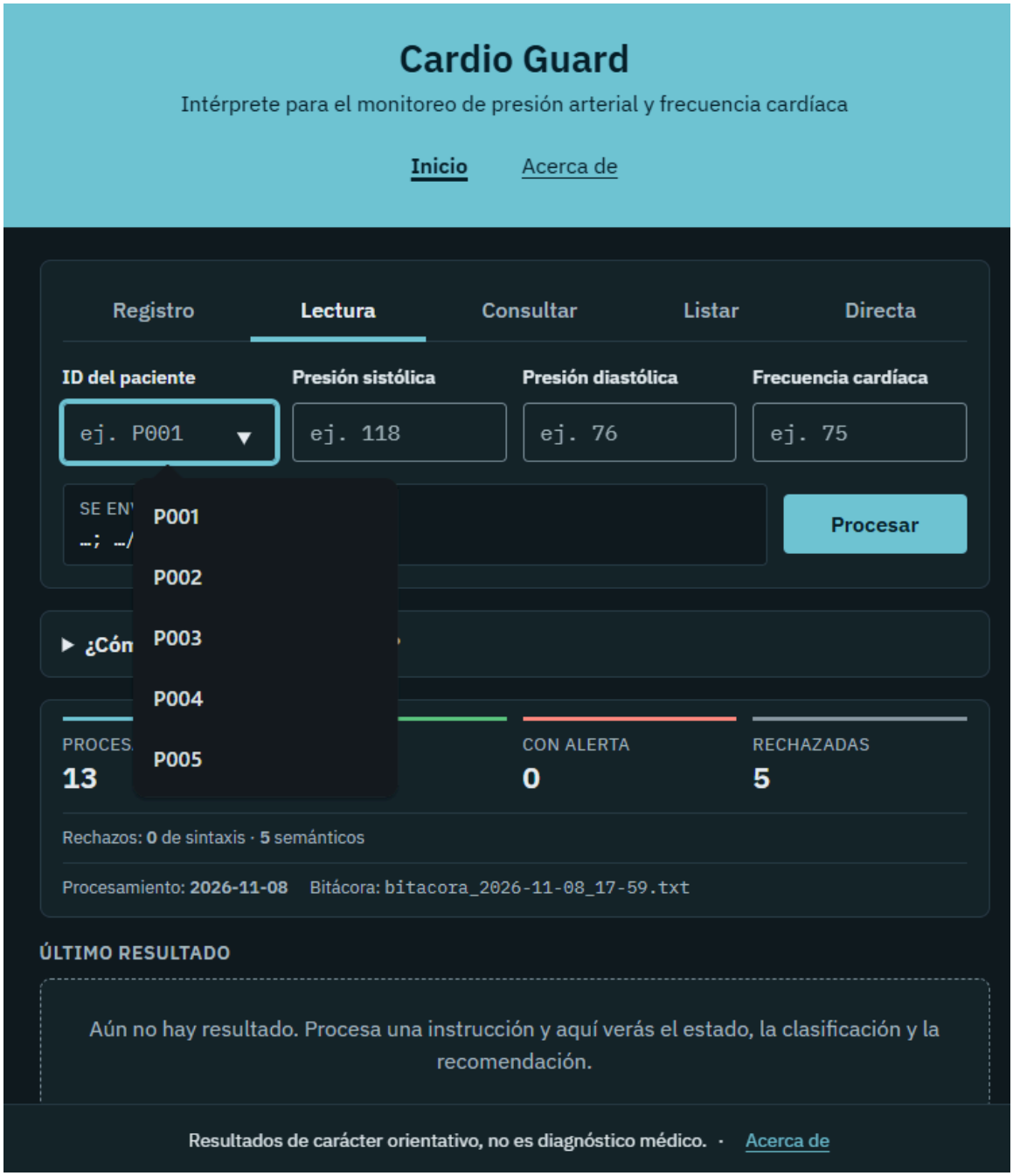


Figura 7.3. Lista de pacientes sugeridos en el campo ID de la pestaña Lectura (captura 13).

7.3 Tarjeta de resultado y estados

Cada instrucción termina en uno de cuatro estados. El estado nunca se comunica solo con color: siempre lleva color, icono y texto.

Estado	Color	Cuándo aparece
Aceptado	Verde	La instrucción es válida y quedó registrada.
Alerta	Rojo (encabezado relleno)	Crisis hipertensiva o frecuencia cardíaca severa. Muestra «¡ALERTA ROJA!» y el protocolo de emergencia.
Advertencia	Naranja	La lectura se guardó, pero tiene un dato sospechoso que conviene verificar.
Rechazado	Gris	Error léxico, sintáctico o semántico. No modifica ningún dato.

El rojo se reserva solo para las alertas; los errores de captura se muestran en gris. En una lectura aceptada, la tarjeta muestra la presión, la frecuencia, la clasificación según la NOM-030-SSA2-2017, la comparación con la última revisión y una recomendación. En un rechazo de sintaxis o léxico, la tarjeta muestra la instrucción con un marcador `^` debajo del punto exacto de la falla y el motivo.

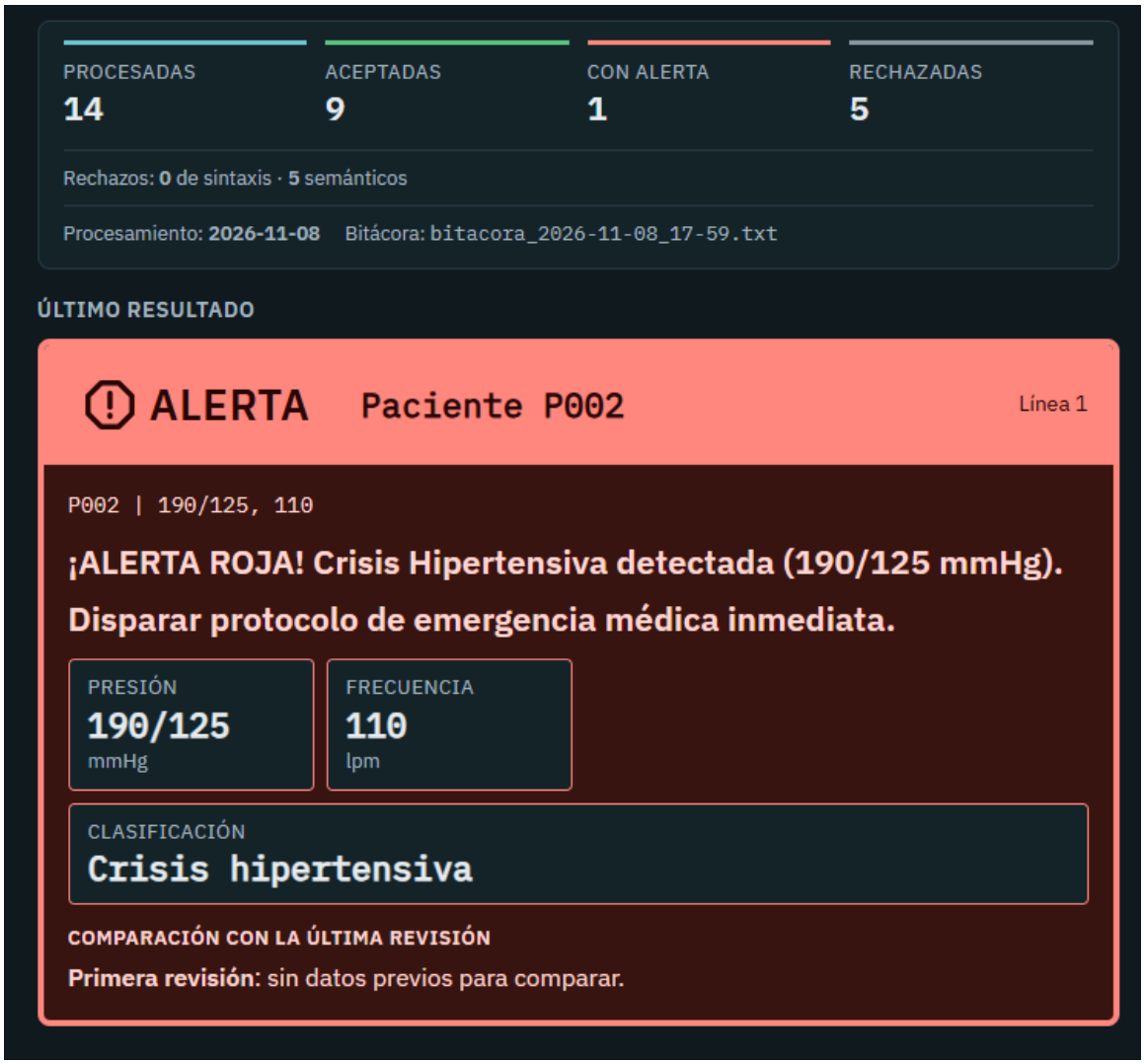


Figura 7.4. Lectura con alerta roja por crisis hipertensiva (captura 14).



Figura 7.5. Lectura aceptada con advertencia por dato sospechoso (captura 18).

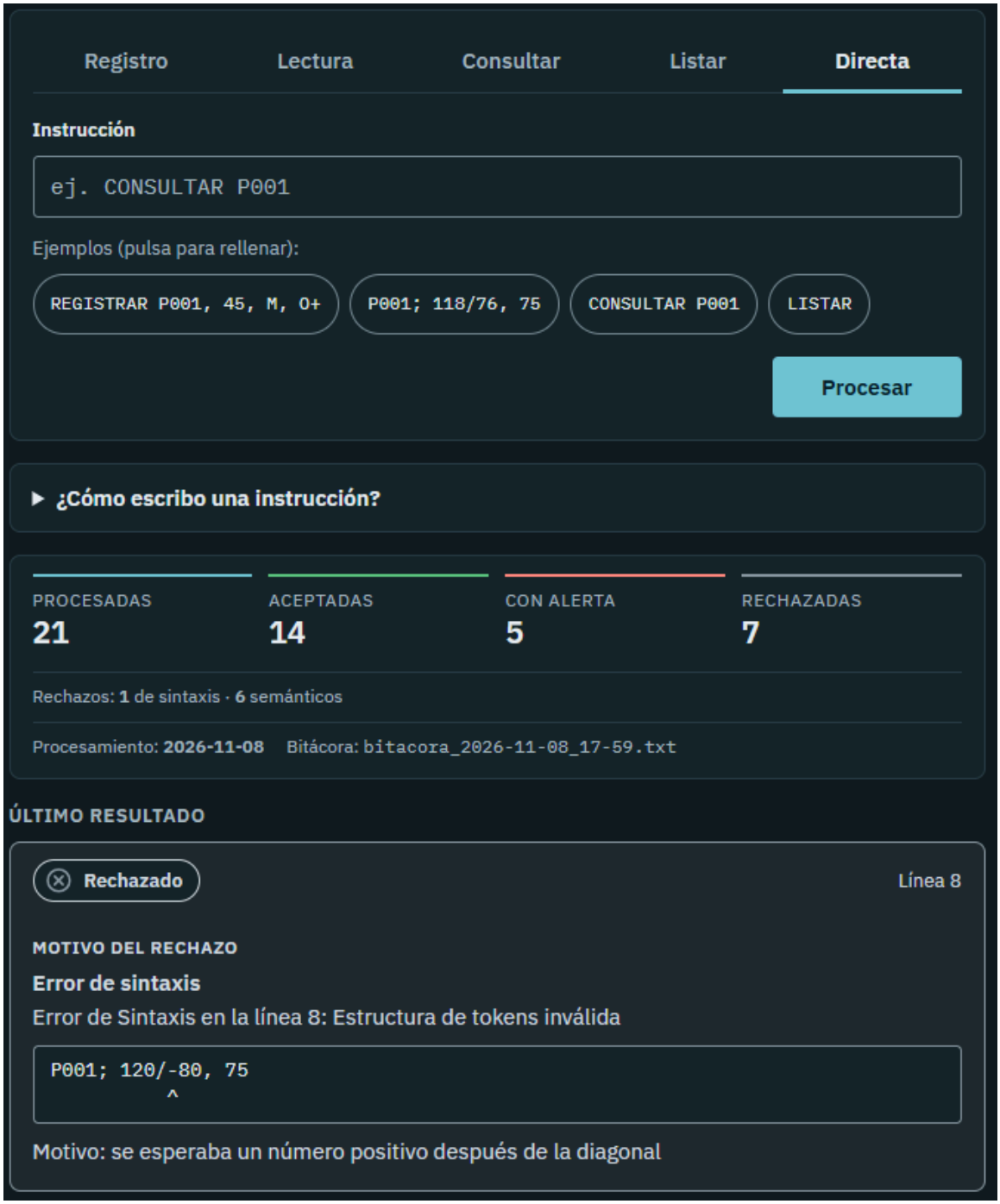


Figura 7.6. Rechazo de sintaxis con el marcador ^ bajo el error (captura 21).

7.4 Consulta y listado de pacientes

`CONSULTAR` muestra la ficha de un paciente: edad, sexo, tipo de sangre, la última revisión con su clasificación y, cuando los datos están en SQL Server, un historial de hasta 10 revisiones. `LISTAR` muestra una tabla con todos los pacientes, su última revisión con fecha y hora y su clasificación con etiquetas clínicas como **CRISIS HIPERTENSIVA** o **DATO SOSPECHOSO**. Ninguna de las dos modifica los datos.



Figura 7.7. Ficha de paciente con historial de revisiones (captura 26).

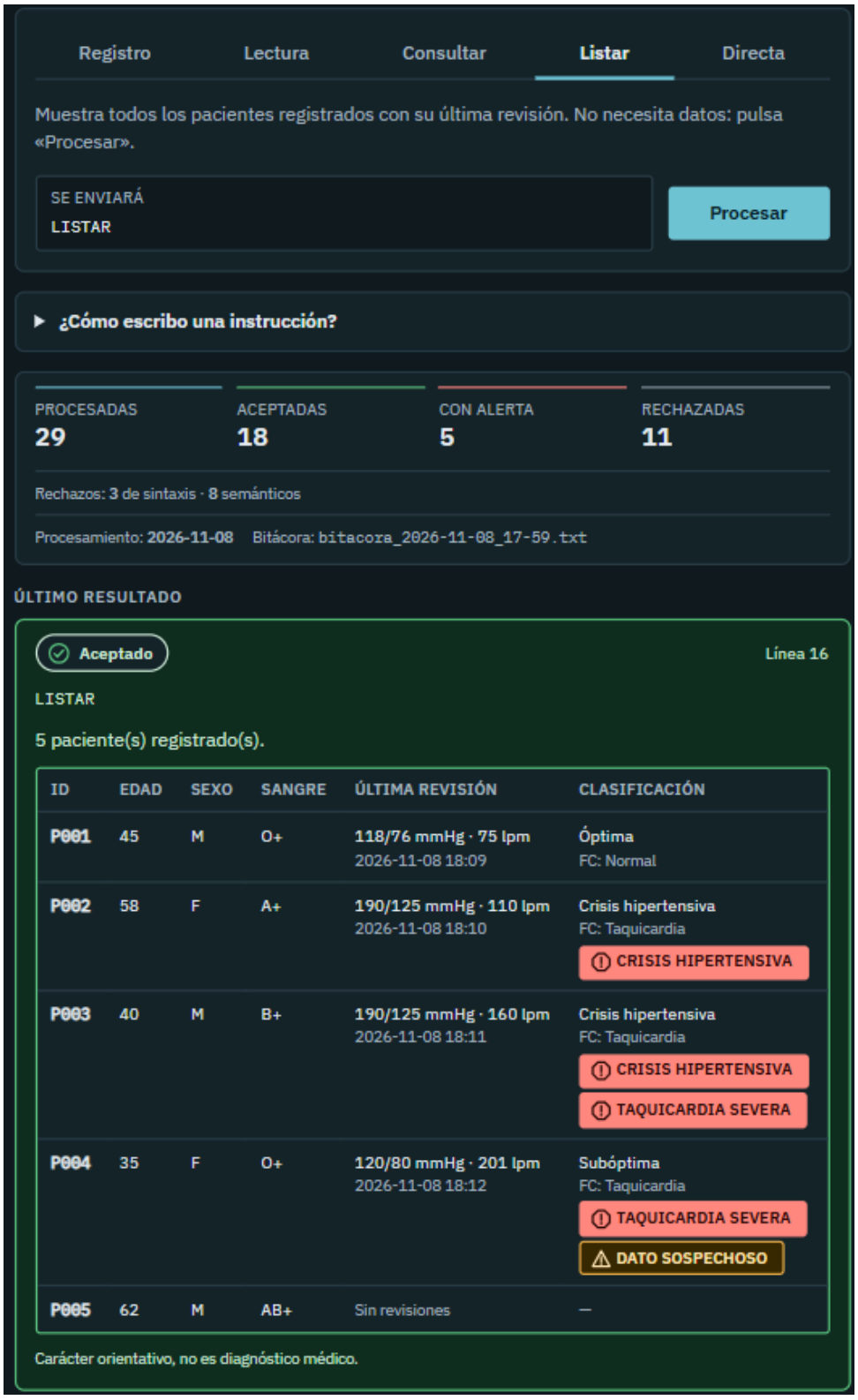


Figura 7.8. Listado de pacientes con clasificación y etiquetas (captura 29).

7.5 Bitácora de la sesión

Debajo del último resultado aparece una tarjeta por cada instrucción enviada, la más reciente arriba, con su número de línea, su estado y sus detalles plegables. Los filtros **Todas, Aceptadas, Alertas, Advertencias y Rechazadas** muestran solo las instrucciones de ese estado. Esta bitácora es la vista en pantalla del archivo de auditoría que el servidor guarda en disco.

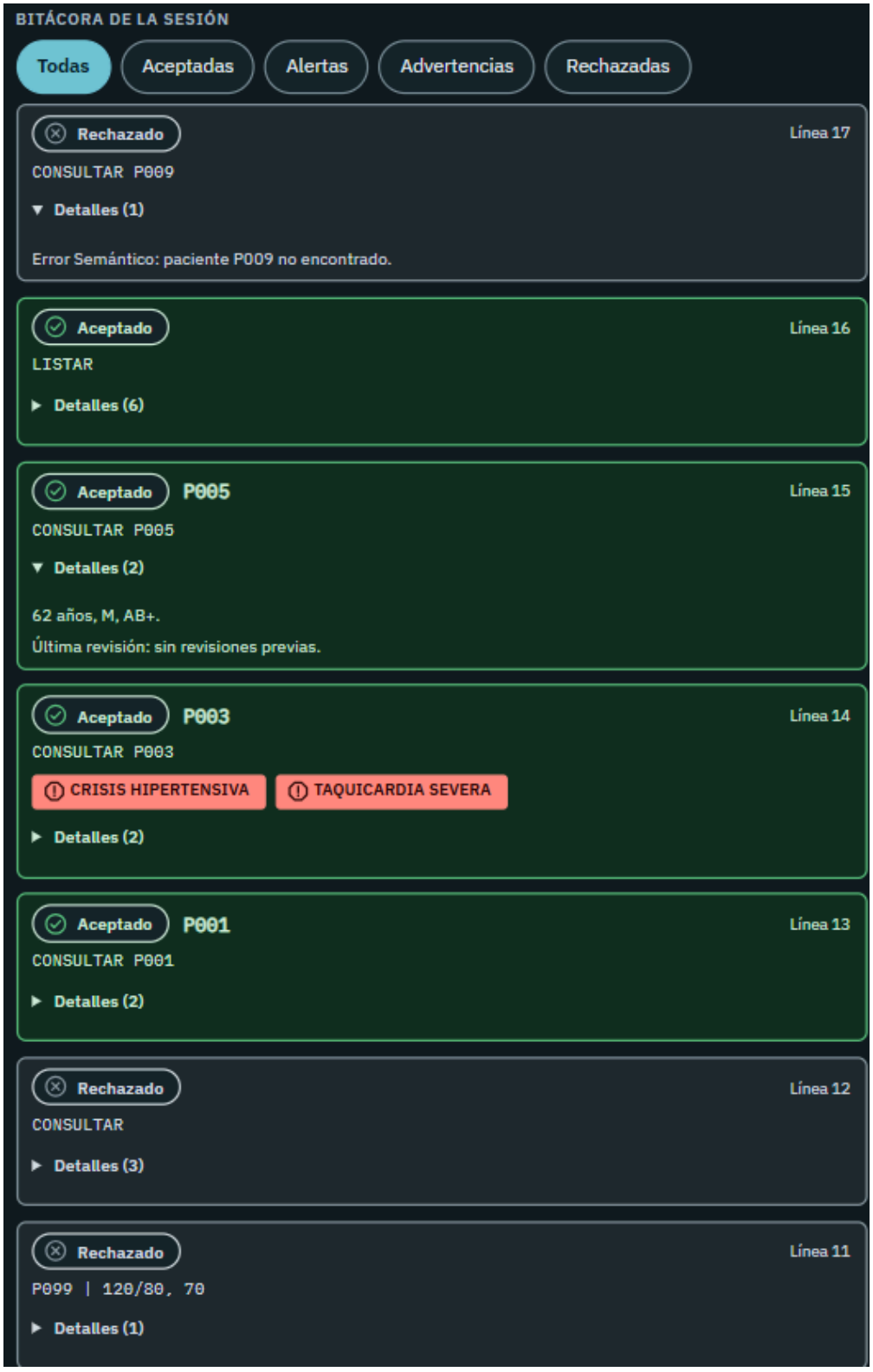


Figura 7.9. Bitácora de la sesión con el filtro Todas (captura 31).

7.6 Ayuda integrada y pantalla Acerca de

La sección plegable «¿Cómo escribo una instrucción?» explica el formato de las cuatro instrucciones con ejemplos, cómo leer un error con el marcador `^`, la diferencia entre el estado de la instrucción y la etiqueta clínica del paciente, y los umbrales de alerta y de dato sospechoso. La pantalla **Acerca de** describe el proyecto, la materia, los integrantes, el profesor y las cuatro etapas del intérprete.

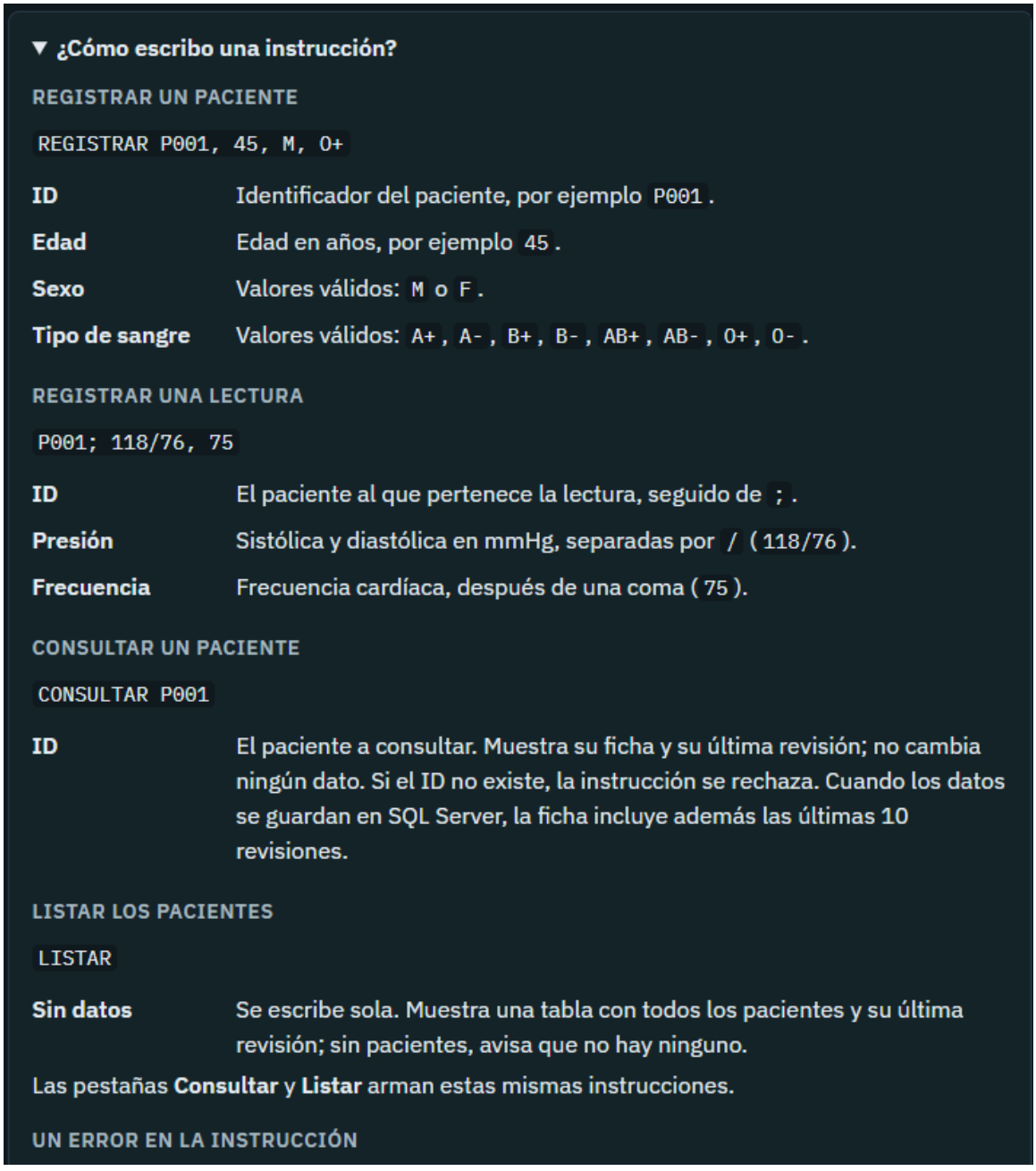


Figura 7.10. Ayuda integrada: formato de las instrucciones (captura 35a).

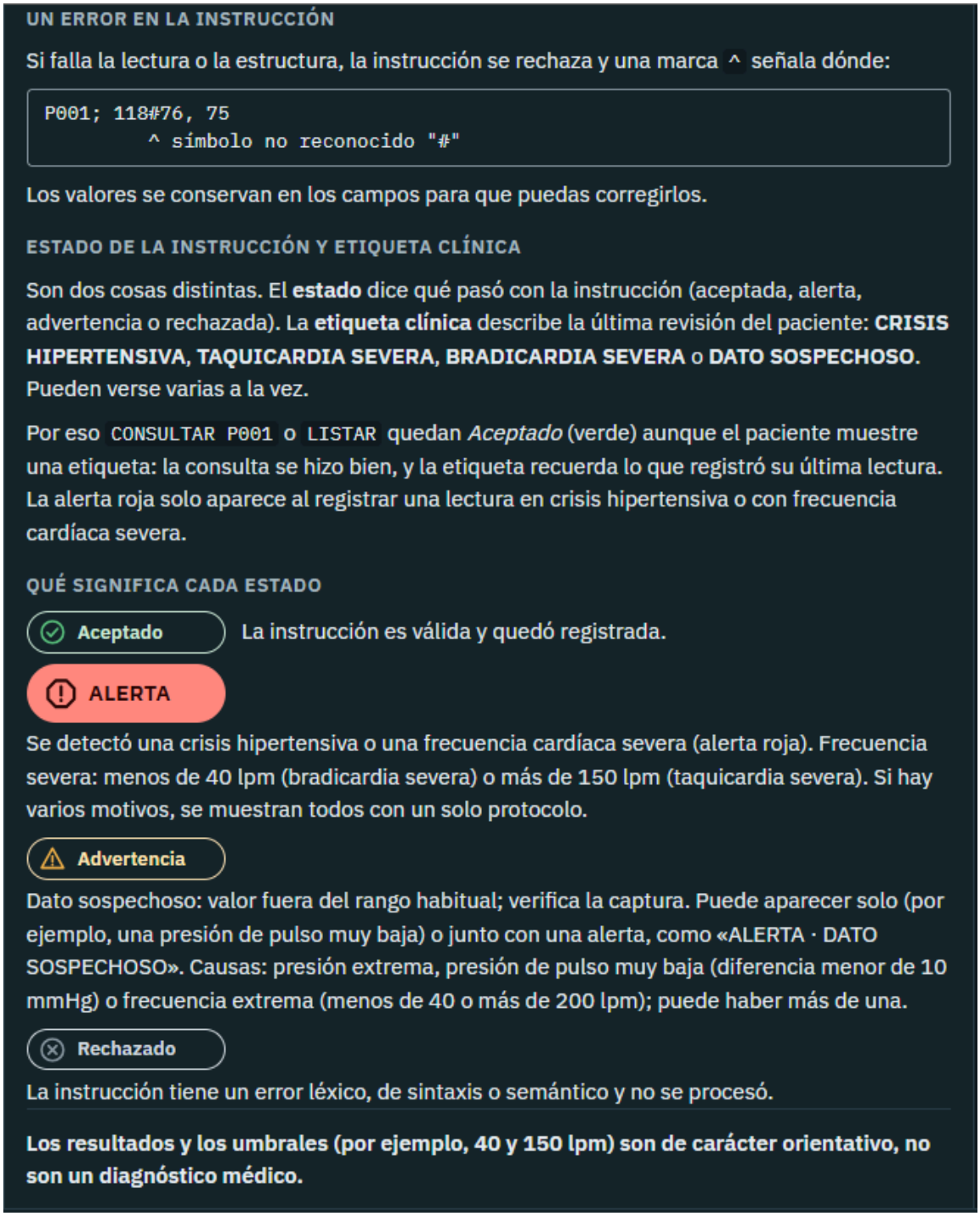


Figura 7.11. Ayuda integrada: estados y etiquetas clínicas (captura 35b).

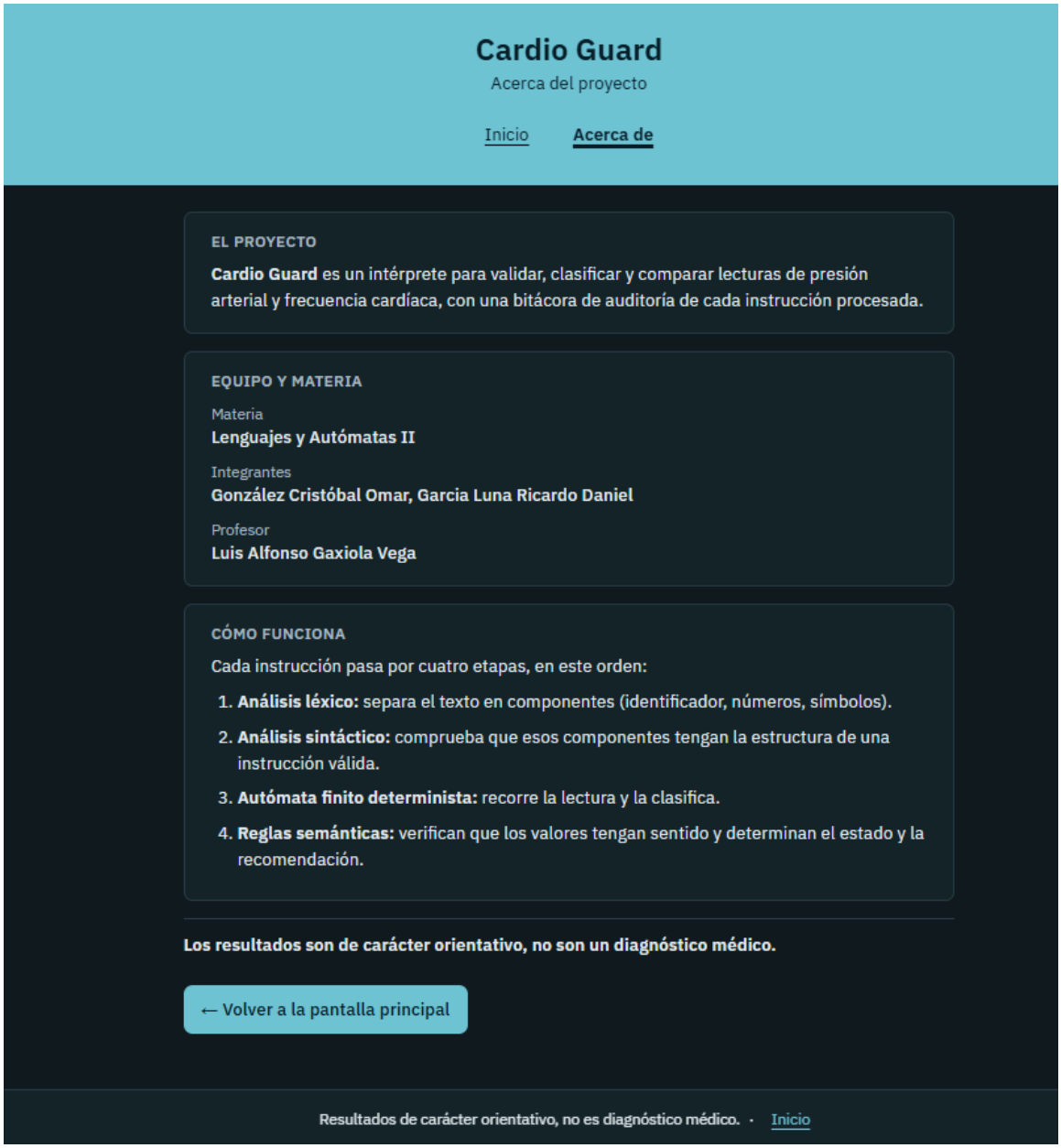


Figura 7.12. Pantalla Acerca de (captura 36).

7.7 Tema claro y oscuro, y vista móvil

La interfaz sigue la preferencia de color del sistema operativo: muestra el tema claro o el oscuro sin necesidad de un botón. El diseño se adapta a pantallas de teléfono desde 375 px de ancho: el formulario se reacomoda en dos columnas, los contadores pasan de cuatro a dos columnas y la tabla de `LISTAR` se convierte en tarjetas apiladas, sin desplazamiento lateral.

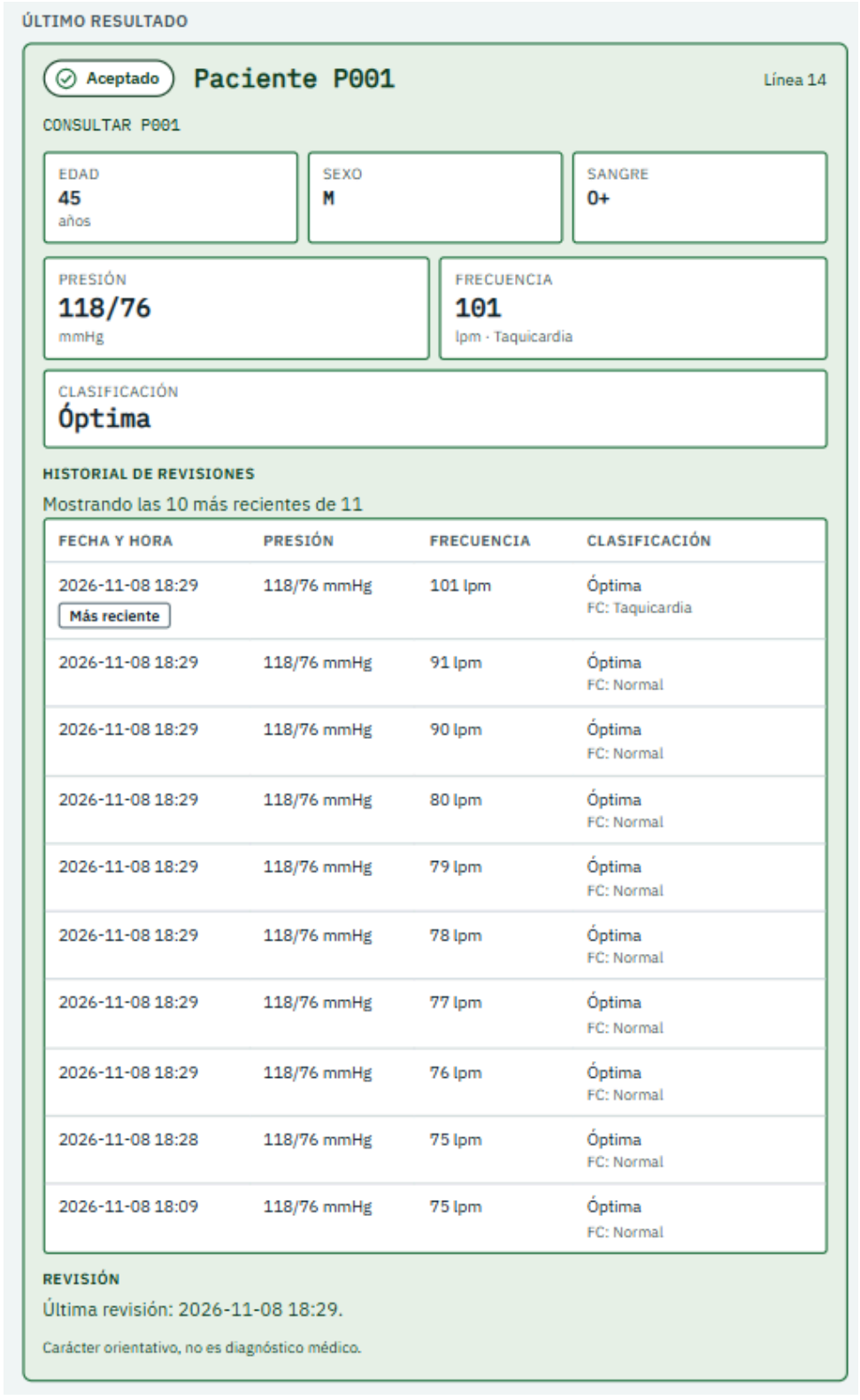


Figura 7.13. Interfaz en tema claro: historial de más de 10 revisiones (captura A1).

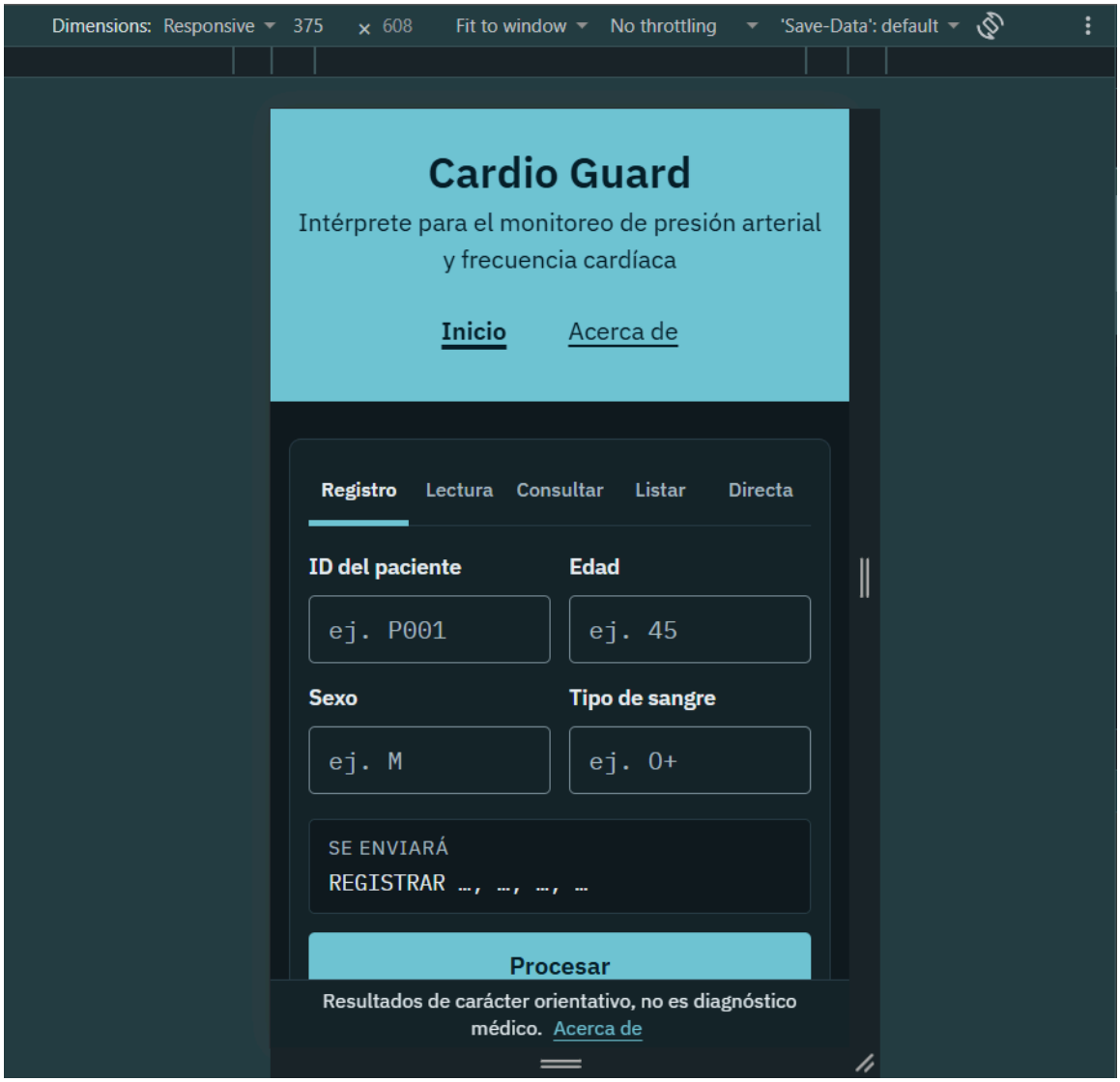


Figura 7.14. Vista móvil a 375 px: pantalla inicial (captura 37).



Figura 7.15. Vista móvil a 375 px: alerta roja (captura 38).

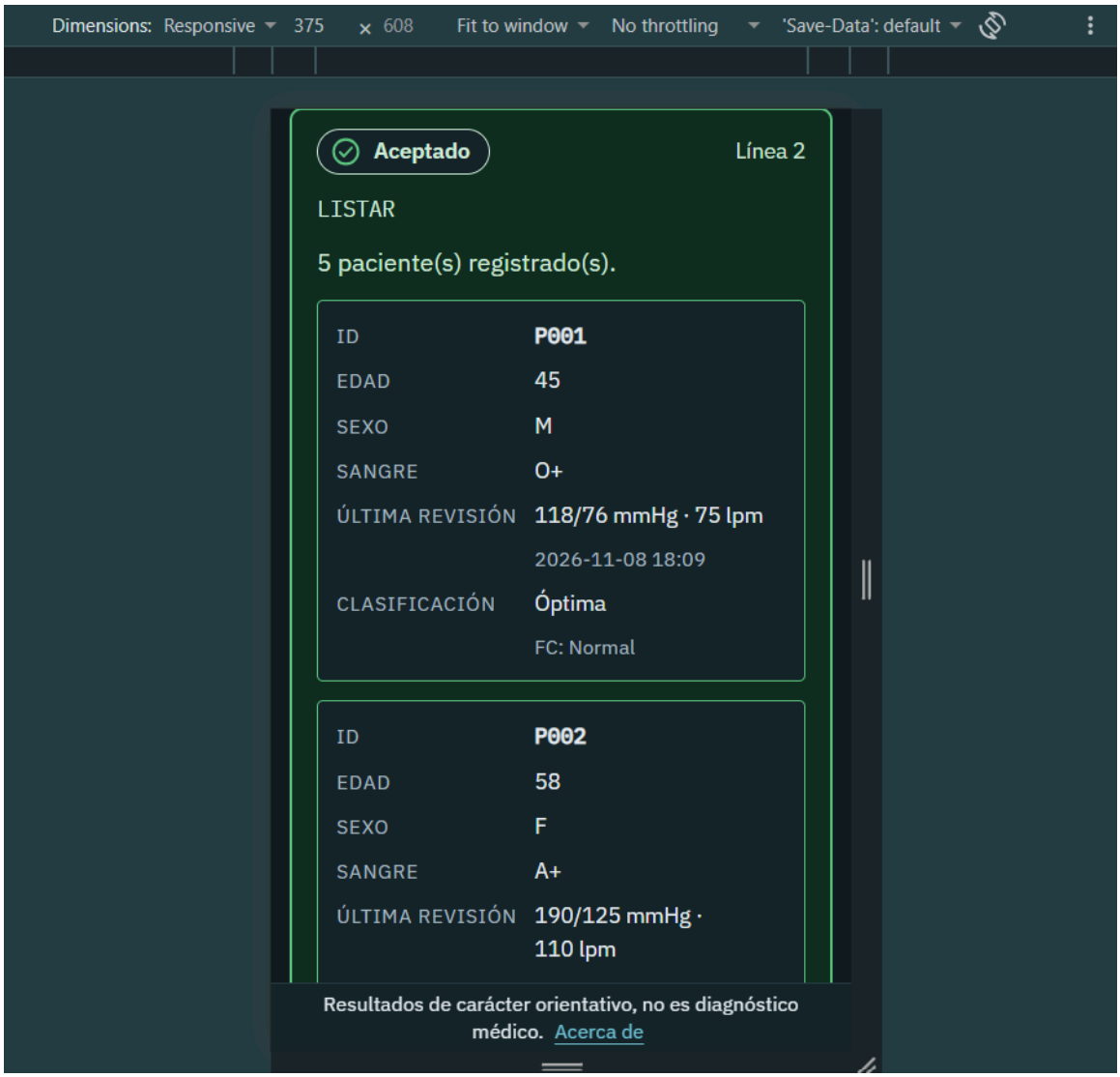


Figura 7.16. Vista móvil a 375 px: listado en tarjetas (captura 39).

8. Código generado en el lenguaje realizado

El intérprete está escrito en JavaScript (Node.js) y suma unas 2,100 líneas en `src/`, más la interfaz en `public/`. Esta sección muestra los fragmentos que implementan cada fase del compilador. El código completo está en el repositorio del proyecto (https://github.com/23212707-omar/cardio-guard) y los módulos del analizador (lexer, parser y AFD) se incluyen completos en el Anexo B.

Archivo	Líneas	Función
`src/lexer/lexer.js`	118	Análisis léxico: reglas de tokens y `analizarLexico()`
`src/parser/parser.js`	216	Análisis sintáctico descendente recursivo
`src/automaton/afd.js`	161	Estados, eventos y tabla de transiciones del AFD
`src/semantics/validarRegistro.js`	75	Reglas semánticas del registro
`src/semantics/validarLectura.js`	63	Reglas semánticas de la lectura
`src/semantics/validarConsulta.js`	27	Regla semántica de la consulta
`src/semantics/clasificacion.js`	266	Clasificación NOM-030, alertas, dato sospechoso y comparación
`src/interpreter.js`	425	Orquestación de las etapas y del AFD
`src/server.js`	176	Servidor Express y API
`src/storage/*.js`	553	Repositorio CSV, repositorio SQL Server y bitácora
`public/app.js`	957	Lógica de la interfaz web

8.1 Analizador léxico (`lexer.js`)

Las reglas se prueban en orden; la primera expresión regular que coincide define el token. Los espacios se saltan y un símbolo sin regla lanza `ErrorLexico` con su posición.

```
const REGLAS = [
  { tipo: TIPO.REGISTRAR, regex: /^REGISTRAR\b/ },
  { tipo: TIPO.CONSULTAR, regex: /^CONSULTAR\b/ },
  { tipo: TIPO.LISTAR, regex: /^LISTAR\b/ },
  { tipo: TIPO.ID, regex: /^P\d+/ },
  { tipo: TIPO.SANGRE, regex: /^[A-Za-z]+[+-]/ },
  { tipo: TIPO.NUM, regex: /^\d+/ },
  { tipo: TIPO.LETRA, regex: /^[A-Za-z]+/ },
  { tipo: TIPO.SLASH, regex: /^\// },
  { tipo: TIPO.COMA, regex: /^,/ },
  { tipo: TIPO.PUNTOYCOMA, regex: /^;/ },
  { tipo: TIPO.GUION, regex: /^-/ },
];

function analizarLexico(linea) {
  const tokens = [];
  let resto = linea;
  let posicion = 0;

  while (resto.length > 0) {
    // Los espacios en blanco se ignoran y no generan token.
    const espacio = resto.match(/^\s+/);
    if (espacio) {
      posicion += espacio[0].length;
      resto = resto.slice(espacio[0].length);
      continue;
    }

    const coincidencia = REGLAS
      .map((regla) => ({ regla, m: resto.match(regla.regex) }))
      .find((r) => r.m !== null);

    if (!coincidencia) {
      throw new ErrorLexico(`símbolo no reconocido "${resto[0]}"`, posicion);
    }

    const { regla, m } = coincidencia;
    tokens.push({ tipo: regla.tipo, valor: m[0], posicion });
    posicion += m[0].length;
    resto = resto.slice(m[0].length);
  }

  tokens.push({ tipo: TIPO.EOF, valor: '', posicion });
  return tokens;
}
```

8.2 Analizador sintáctico (`parser.js`)

Cada regla de la gramática es una función. `esperar()` exige el token que corresponde y, si no está, lanza `ErrorSintactico`. La regla inicial `S` se decide con el primer token.

```
// LEC -> id ; P , F
function parsearLEC() {
  const id = esperar(TIPO.ID, () => 'se esperaba el identificador del
paciente').valor;
```

```
  esperar(TIPO.PUNTOYCOMA, () => 'se esperaba ";"');
  const presion = parsearP();
  esperar(TIPO.COMA, () => 'se esperaba ","');
  const frecuencia = parsearF();
  exigirFinDeLinea();

  return {
    tipo: 'LEC',
    id,
    sistolica: presion.sistolica,
    diastolica: presion.diastolica,
    frecuencia,
  };
}

// P -> num / num
function parsearP() {
  const numIzq = esperar(TIPO.NUM, () => 'se esperaba la presión sistólica').valor;
  esperar(TIPO.SLASH, () => 'se esperaba "/"');

  // Caso documentado: un guion justo después de la diagonal ("120/-80").
  if (actual().tipo === TIPO.GUION) {
    throw new ErrorSintactico(
      'se esperaba un número positivo después de la diagonal',
      actual().posicion
    );
  }

  const numDer = esperar(TIPO.NUM, () => 'se esperaba la presión diastólica').valor;
  return { sistolica: Number(numIzq), diastolica: Number(numDer) };
}

// S -> REG | LEC | CON | LIS   (se decide con el primer token, sin retroceder)
const primero = actual();
if (primero.tipo === TIPO.REGISTRAR) return parsearREG();
if (primero.tipo === TIPO.CONSULTAR) return parsearCON();
if (primero.tipo === TIPO.LISTAR) return parsearLIS();
if (primero.tipo === TIPO.ID) return parsearLEC();
throw new ErrorSintactico(
  'se esperaba REGISTRAR, CONSULTAR, LISTAR o el identificador de un paciente',
  primero.posicion
);
```

8.3 Autómata finito determinista (`afd.js`)

La función de transición δ es una tabla cuyas claves combinan estado y evento. Si la combinación no existe, el autómata se detiene con `TransicionInvalidaError`.

```
// δ: función de transición, como tabla "estado:evento" -> estado siguiente.
const TRANSICIONES = Object.freeze({
  [`${ESTADO.INICIO}:${EVENTO.LEXICO_SINTACTICO_ERROR}`]: ESTADO.RECHAZADO,
  [`${ESTADO.INICIO}:${EVENTO.LEXICO_SINTACTICO_OK_REG}`]:
ESTADO.VALIDANDO_SEMANTICA_REGISTRO,
  [`${ESTADO.INICIO}:${EVENTO.LEXICO_SINTACTICO_OK_LEC}`]: ESTADO.BUSCANDO_PACIENTE,
  // ... (18 transiciones en total, ver tabla de la sección 6.7)
```

```
  [`${ESTADO.CLASIFICANDO}:${EVENTO.CLASIFICACION_ALERTA_Y_ADVERTENCIA}`]:
    ESTADO.ACEPTADO_CON_ALERTA_Y_ADVERTENCIA,
});

class AFD {
  constructor() {
    this.estado = ESTADO.INICIO;
    this.historial = [ESTADO.INICIO];
  }

  /** Aplica un evento y mueve el autómata al siguiente estado. */
  transitar(evento) {
    const clave = `${this.estado}:${evento}`;
    const siguiente = TRANSICIONES[clave];
    if (siguiente === undefined) {
      throw new TransicionInvalidaError(this.estado, evento);
    }
    this.estado = siguiente;
    this.historial.push(siguiente);
    return this.estado;
  }

  esAceptado() {
    return ESTADOS_DE_ACEPTACION.has(this.estado);
  }

  esRechazado() {
    return this.estado === ESTADO.RECHAZADO;
  }
}
```

8.4 Análisis semántico de la lectura (`validarLectura.js`)

```
function validarLectura(arbolLec) {
  const { sistolica, diastolica, frecuencia } = arbolLec;

  if (sistolica <= diastolica) {
    return {
      ok: false,
      mensaje: `Error Semántico: la sistólica (${sistolica}) debe ser mayor que la
diastólica (${diastolica}).`,
    };
  }

  if (sistolica <= 0 || diastolica <= 0 || frecuencia <= 0) {
    return { ok: false, mensaje: 'Error Semántico: los valores no pueden ser cero o
negativos.' };
  }

  if (frecuencia < 30 || frecuencia > 220) {
    return {
      ok: false,
      mensaje: `Error Semántico: frecuencia cardíaca fuera de rango válido
(${frecuencia}).`,
    };
```

```
  }

  // Presión extrema, pulso muy bajo o frecuencia en extremos: no se rechaza,
  // se marca como "dato sospechoso" para que la clasificación decida el aviso.
  const causasSospechoso = causasDatoSospechoso(sistolica, diastolica, frecuencia);
  return { ok: true, datoSospechoso: causasSospechoso.length > 0, causasSospechoso };
}
```

8.5 Clasificación NOM-030 y comparación (`clasificacion.js`)

Las categorías se recorren de la más grave a la menos grave; la primera que coincide es la clasificación. Así se aplica sola la regla de tomar la categoría más alta entre sistólica y diastólica.

```
const CATEGORIAS_PRESION = [
  { nombre: 'Crisis hipertensiva',  nivel: 6, coincide: (s, d) => s > 180 || d > 120
},
  { nombre: 'Hipertensión grado 3', nivel: 5, coincide: (s, d) => s >= 180 || d >=
110 },
  { nombre: 'Hipertensión grado 2', nivel: 4, coincide: (s, d) => s >= 160 || d >=
100 },
  { nombre: 'Hipertensión grado 1', nivel: 3, coincide: (s, d) => s >= 140 || d >= 90
},
  { nombre: 'Limítrofe',            nivel: 2, coincide: (s, d) => s >= 130 || d >= 85
},
  { nombre: 'Subóptima',            nivel: 1, coincide: (s, d) => s >= 120 || d >= 80
},
  { nombre: 'Óptima',               nivel: 0, coincide: () => true },
];

function clasificarPresion(sistolica, diastolica) {
  const categoria = CATEGORIAS_PRESION.find((c) => c.coincide(sistolica,
diastolica));
  return { nombre: categoria.nombre, nivel: categoria.nivel };
}

function compararConUltimaRevision(actual, anterior) {
  if (anterior === null) return { resultado: 'PRIMERA_REVISION' };

  // ... se clasifican la lectura actual y la anterior y se comparan sus niveles

  // Regla conservadora: cualquier empeoramiento domina sobre una mejora.
  let resultado;
  if (cambioPresion === 'EMPEORO' || cambioFrecuencia === 'EMPEORO') {
    resultado = 'NEGATIVO';
  } else if (cambioPresion === 'MEJORO' || cambioFrecuencia === 'MEJORO') {
    resultado = 'POSITIVO';
  } else {
    resultado = 'SIN_CAMBIO';
  }
  // ...
}
```

8.6 Intérprete (`interpreter.js`)

`procesarLinea()` une todas las fases: crea un AFD, ejecuta el análisis léxico y el sintáctico, y según el tipo de instrucción manda el control a la función que hace la validación semántica, la clasificación y el guardado.

```
async function procesarLinea(textoOriginal, numeroLinea) {
  const afd = crearAFD();

  // --- 1 y 2. Léxico y sintáctico ---
  let tokens;
  try {
    tokens = analizarLexico(textoOriginal);
  } catch (err) {
    if (!(err instanceof ErrorLexico)) throw err;
    afd.transitar(EVENTO.LEXICO_SINTACTICO_ERROR);
    return rechazarPorEntrada(afd, numeroLinea, textoOriginal, 'LEXICO',
      `Error Léxico en la línea ${numeroLinea}: Símbolo no reconocido`, err);
  }

  let arbol;
  try {
    arbol = analizarSintactico(tokens);
  } catch (err) {
    if (!(err instanceof ErrorSintactico)) throw err;
    afd.transitar(EVENTO.LEXICO_SINTACTICO_ERROR);
    return rechazarPorEntrada(afd, numeroLinea, textoOriginal, 'SINTACTICO',
      `Error de Sintaxis en la línea ${numeroLinea}: Estructura de tokens inválida`,
err);
  }

  switch (arbol.tipo) {
    case 'REG':
      afd.transitar(EVENTO.LEXICO_SINTACTICO_OK_REG);
      return await procesarRegistro(afd, arbol, numeroLinea);
    case 'CON':
      afd.transitar(EVENTO.LEXICO_SINTACTICO_OK_CON);
      return await procesarConsulta(afd, arbol, numeroLinea);
    case 'LIS':
      afd.transitar(EVENTO.LEXICO_SINTACTICO_OK_LIS);
      return await procesarListado(afd, numeroLinea);
    default:
      afd.transitar(EVENTO.LEXICO_SINTACTICO_OK_LEC);
      return await procesarLectura(afd, arbol, numeroLinea);
  }
}
```

8.7 Salida del análisis

Ejecutando el lexer y el parser sobre instrucciones reales se obtiene lo siguiente (token, lexema entre paréntesis y posición):

```
REGISTRAR P001, 45, M, O+
  REGISTRAR(REGISTRAR)@0 ID(P001)@10 COMA@14 NUM(45)@16 COMA@18 LETRA(M)@20 COMA@21
SANGRE(O+)@23 EOF@25
```

```
  -> {"tipo":"REG","id":"P001","edad":45,"sexo":"M","sangre":"O+"}

P001; 118/76, 75
  ID(P001)@0 PUNTOYCOMA@4 NUM(118)@6 SLASH@9 NUM(76)@10 COMA@12 NUM(75)@14 EOF@16
  -> {"tipo":"LEC","id":"P001","sistolica":118,"diastolica":76,"frecuencia":75}

CONSULTAR P001
  CONSULTAR(CONSULTAR)@0 ID(P001)@10 EOF@14

LISTAR
  LISTAR(LISTAR)@0 EOF@6

P001; 120#80, 75
  ErrorLexico: símbolo no reconocido "#" (posición 9)

P001; 120/-80, 75
  ErrorSintactico: se esperaba un número positivo después de la diagonal (posición
10)
```

9. Manual de usuario

Este manual explica, paso a paso, cómo registrar pacientes, capturar lecturas, consultar resultados y entender cada mensaje de Cardio Guard. Está dirigido al personal que captura las lecturas; la instalación y configuración del sistema están en el Manual de administrador.

Los resultados son de carácter orientativo: Cardio Guard no sustituye la valoración de un médico.

9.1 Antes de empezar

1. Pida al administrador que encienda el servidor de Cardio Guard.
2. Abra un navegador web (Chrome, Edge o Firefox) y entre a **http://localhost:3000**.
3. Verá la pantalla inicial con las cinco pestañas de captura (sección 7, figura 7.1).

Tres cosas que conviene saber desde el inicio:

- **Cada instrucción recibe un número de línea** (Línea 1, Línea 2…). La numeración vuelve a empezar en 1 cada vez que se recarga la página. Los contadores del resumen **no** se reinician, porque los lleva el servidor.
- **Una instrucción rechazada no cambia ningún dato.** Se puede corregir y volver a enviar.
- **El estado de la instrucción y la etiqueta clínica son cosas distintas.** El estado (Aceptado, Alerta, Advertencia, Rechazado) dice qué pasó con la instrucción. La etiqueta clínica (por ejemplo, CRISIS HIPERTENSIVA) describe la última revisión del paciente.

9.2 Registrar un paciente

Un paciente debe estar registrado antes de capturar sus lecturas.

1. Abra la pestaña **Registro**.
2. Escriba el **ID del paciente** (la letra P seguida de números, por ejemplo `P001`), la **edad**, el **sexo** y el **tipo de sangre**.
3. Revise el cuadro «Se enviará»: debe mostrar `REGISTRAR P001, 45, M, O+`.
4. Pulse **Procesar**.

Si los datos son válidos, la tarjeta muestra **Aceptado**, «Paciente registrado. Aún no tiene revisiones.» y los datos capturados.

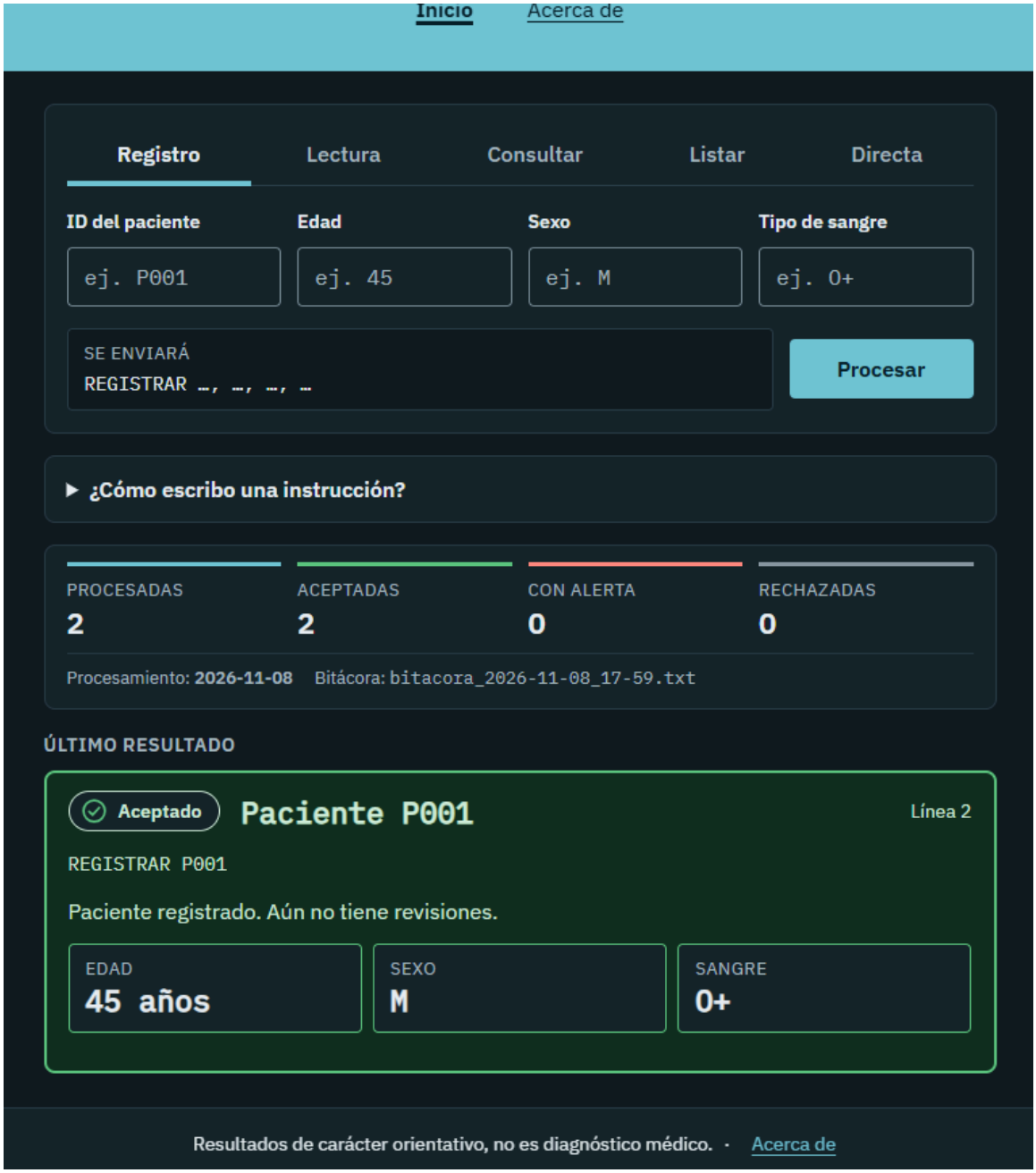


Figura 9.1. Paciente P001 registrado correctamente (captura 05).

El registro se rechaza si algún dato no cumple estas reglas:

Dato	Regla	Mensaje si no se cumple
ID del paciente	Letra P seguida de números, máximo 20 caracteres y no repetido	«identificador de paciente inválido o duplicado» o «demasiado largo, máximo 20 caracteres»
Edad	De 0 a 120 años	«edad fuera de rango válido»

Dato	Regla	Mensaje si no se cumple
Sexo	M o F	«valor de sexo no válido»
Tipo de sangre	A+, A-, B+, B-, AB+, AB-, O+ u O-	«tipo de sangre no reconocido»

Un ID ya registrado no se puede volver a usar, y el intento no modifica al paciente existente.



Figura 9.2. Rechazo por ID duplicado (captura 06).



Figura 9.3. Rechazo por edad fuera de rango (captura 07).

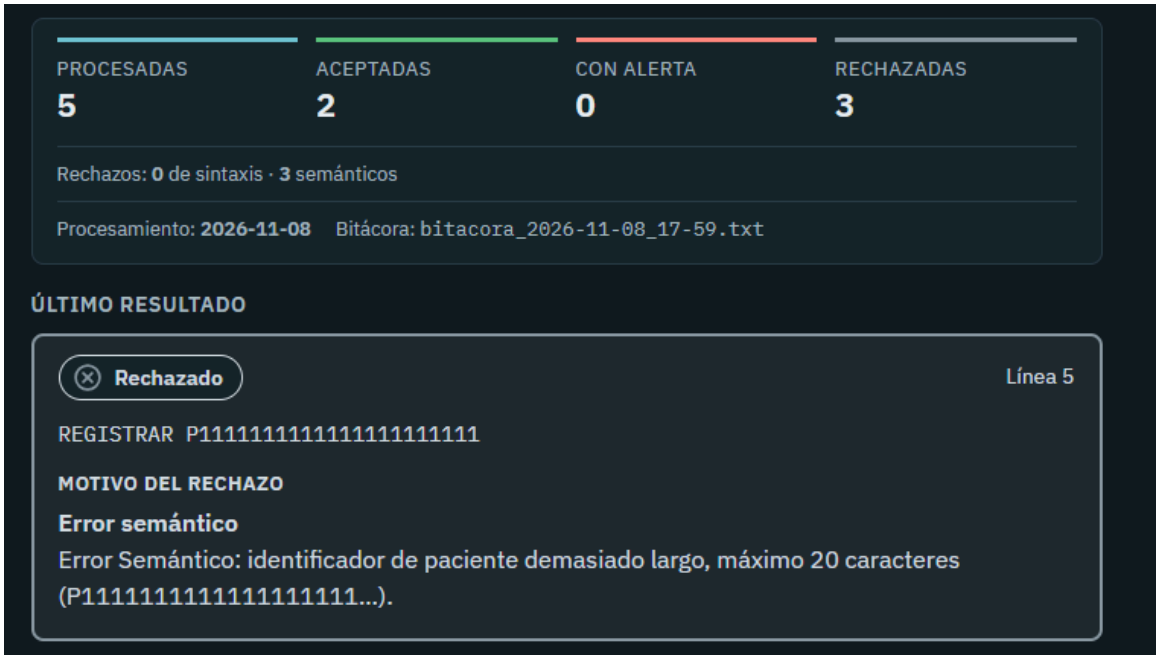


Figura 9.4. Rechazo por ID de más de 20 caracteres (captura 08).



Figura 9.5. Rechazo por sexo no válido (captura 09).



Figura 9.6. Rechazo por tipo de sangre no reconocido (captura 10).

9.3 Registrar una lectura

1. Abra la pestaña **Lectura**.
2. En **ID del paciente**, escriba el ID o elija uno de la lista desplegable de pacientes registrados.
3. Escriba la **presión sistólica**, la **presión diastólica** (ambas en mmHg) y la **frecuencia cardíaca** (en latidos por minuto).
4. Revise el cuadro «Se enviará»: debe mostrar algo como `P001; 118/76, 75`.
5. Pulse **Procesar**.

La tarjeta de una lectura aceptada muestra la presión, la frecuencia con su categoría, la clasificación de la presión, la comparación con la revisión anterior y una recomendación. La fecha y hora de la revisión las asigna el sistema.

La presión se clasifica según la norma oficial mexicana NOM-030-SSA2-2017. Si la sistólica y la diastólica caen en categorías distintas, se usa la más alta.

Clasificación	Sistólica (mmHg)		Diastólica (mmHg)
Óptima	menor de 120	y	menor de 80
Subóptima	120 a 129	o	80 a 84
Limítrofe	130 a 139	o	85 a 89
Hipertensión grado 1	140 a 159	o	90 a 99
Hipertensión grado 2	160 a 179	o	100 a 109
Hipertensión grado 3	180 o más	o	110 o más

Clasificación	Sistólica (mmHg)		Diastólica (mmHg)
Crisis hipertensiva	mayor de 180	o	mayor de 120

La frecuencia cardíaca en reposo se clasifica como **Bradicardia** (menos de 60 lpm), **Normal** (60 a 100 lpm) o **Taquicardia** (más de 100 lpm).

La comparación se hace contra la última revisión del paciente y da uno de tres resultados: **Mejoró**, **Empeoró** o **Sin cambio**. Si la presión o la frecuencia empeoró, el resultado es Empeoró. La comparación es por categoría, no por el número: pasar de 130/85 a 128/84 sigue siendo Sin cambio si ambas lecturas caen en la misma categoría. En la primera lectura de un paciente aparece «Primera revisión: sin datos previos para comparar».

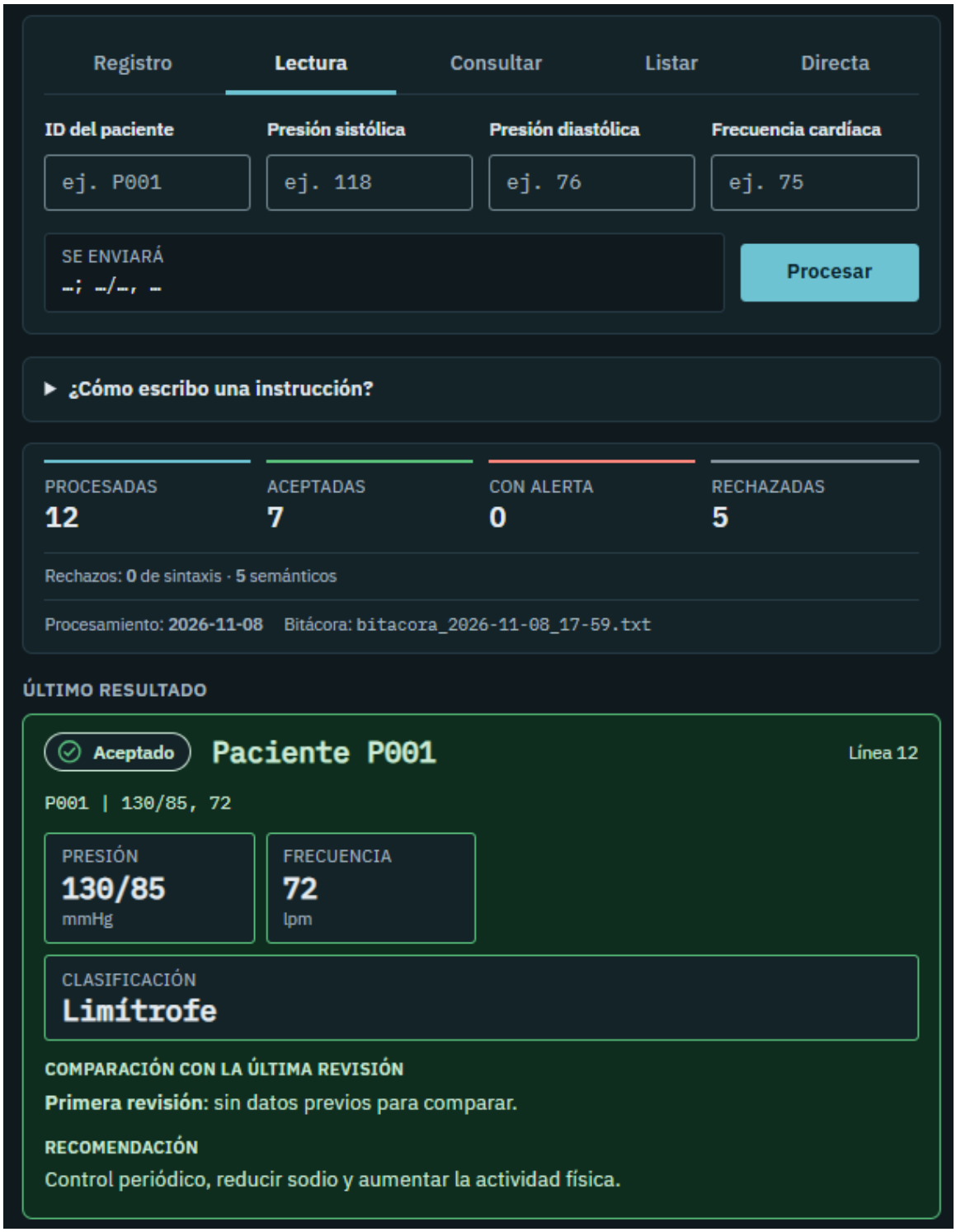


Figura 9.7. Primera lectura de un paciente, clasificada como Limítrofe (captura 11).



Figura 9.8. Segunda lectura con comparación: la presión mejoró (captura 12).

9.4 Alertas y datos sospechosos

Una lectura aceptada puede llegar con una **alerta**, con una **advertencia** o con ambas. En los dos casos la lectura se guarda.

Aviso	Cuándo aparece	Qué hacer
Alerta roja: crisis hipertensiva	Sistólica mayor de 180 o diastólica mayor de 120 mmHg	Seguir el protocolo de emergencia médica.
Alerta roja: taquicardia severa	Frecuencia mayor de 150 lpm	Seguir el protocolo de emergencia médica.
Alerta roja: bradicardia severa	Frecuencia menor de 40 lpm	Seguir el protocolo de emergencia médica.
Advertencia: dato sospechoso	Sistólica mayor de 250 o diastólica mayor de 150; diferencia entre sistólica y diastólica	Verificar que la lectura se haya capturado bien.

Aviso	Cuándo aparece	Qué hacer
	menor de 10 mmHg; o frecuencia menor de 40 o mayor de 200 lpm	

Con una alerta no se muestra recomendación de texto, porque lo que procede es el protocolo de emergencia. Si coinciden varias alertas, cada una aparece en su propia línea y el protocolo se muestra una sola vez.

Una advertencia indica que el **dato** parece un error de captura, no que la lectura sea leve. Por ejemplo, 120/115 es Hipertensión grado 3 por la diastólica y a la vez es dato sospechoso por la diferencia de 5 mmHg; por eso muestra la recomendación «Se requiere valoración médica urgente» junto a la advertencia (sección 7, figura 7.5).



Figura 9.9. Alerta por taquicardia severa: la presión no es el motivo (captura 15).



Figura 9.10. Alerta por bradicardia severa con dato sospechoso (captura 16).

En la figura 9.10 la comparación dice «Frecuencia: sin cambio» aunque el paciente pasó de 151 lpm a 39 lpm. Es correcto: taquicardia y bradicardia tienen el mismo nivel de severidad, y la comparación es por nivel.



Figura 9.11. Dos alertas en la misma lectura y el protocolo mostrado una sola vez (captura 17).



Figura 9.12. Alerta y dato sospechoso por frecuencia extrema (captura 19).

9.5 Errores y rechazos

Una instrucción rechazada aparece en gris, con el tipo de error y su motivo. No modifica ningún dato: basta con corregirla y volver a enviarla.

Tipo de error	Qué significa	Ejemplo
Error léxico	La instrucción tiene un símbolo que no pertenece al lenguaje	`P001; 120#80, 75`

Tipo de error	Qué significa	Ejemplo
Error de sintaxis	Los elementos están en un orden o forma incorrectos	`P001; 120/-80, 75` o `CONSULTAR` sin ID
Error semántico	La forma es correcta, pero los valores no tienen sentido	Sistólica no mayor que la diastólica, frecuencia fuera de 30 a 220 lpm, valores en cero o negativos, o paciente no registrado

En los errores léxicos y de sintaxis, la tarjeta repite la instrucción y coloca un marcador `^` debajo del carácter exacto donde está la falla, con el motivo debajo (sección 7, figura 7.6). Estos errores se capturan más fácil en la pestaña **Directa**, donde se escribe la instrucción completa.



Figura 9.13. Rechazo: la sistólica debe ser mayor que la diastólica (captura 20).

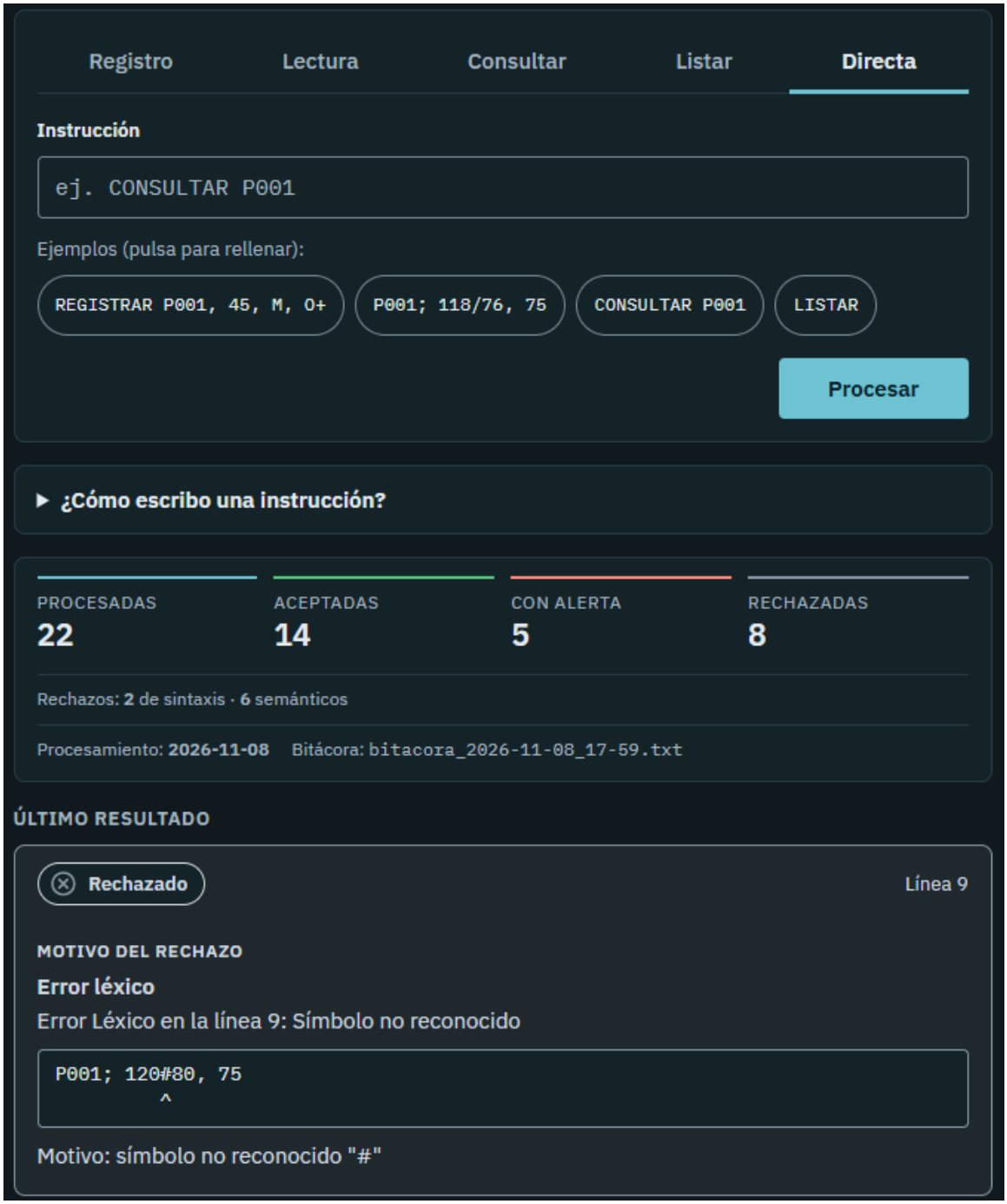


Figura 9.14. Error léxico con el marcador ^ bajo el símbolo # (captura 22).



Figura 9.15. Rechazo por frecuencia cardíaca fuera de rango (captura 23).

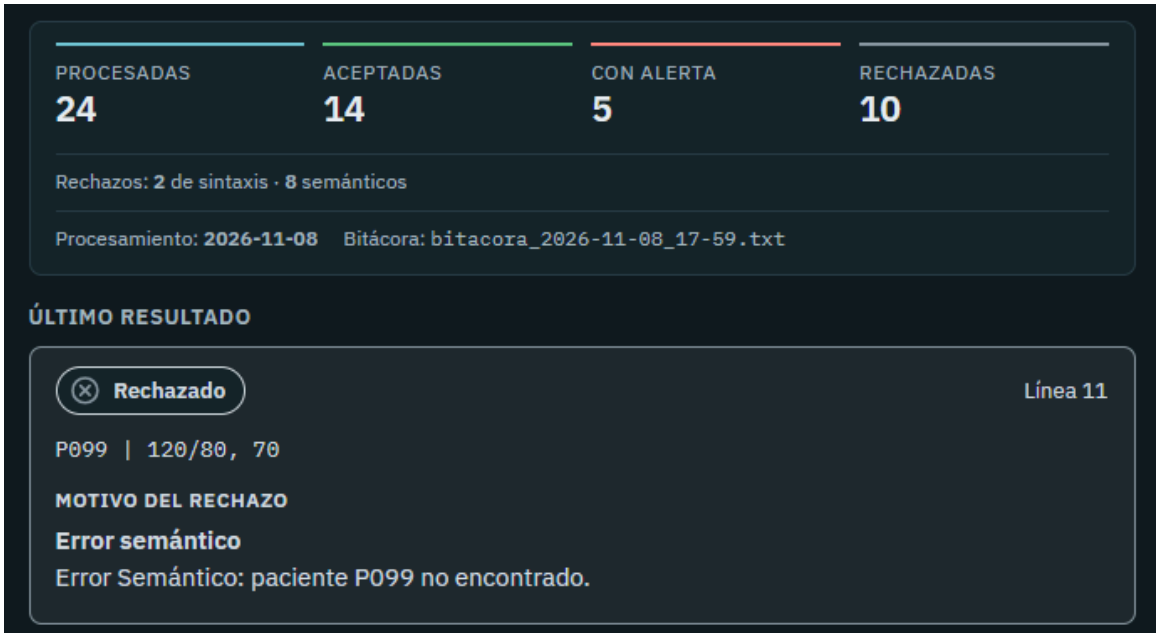


Figura 9.16. Rechazo por paciente no registrado (captura 24).

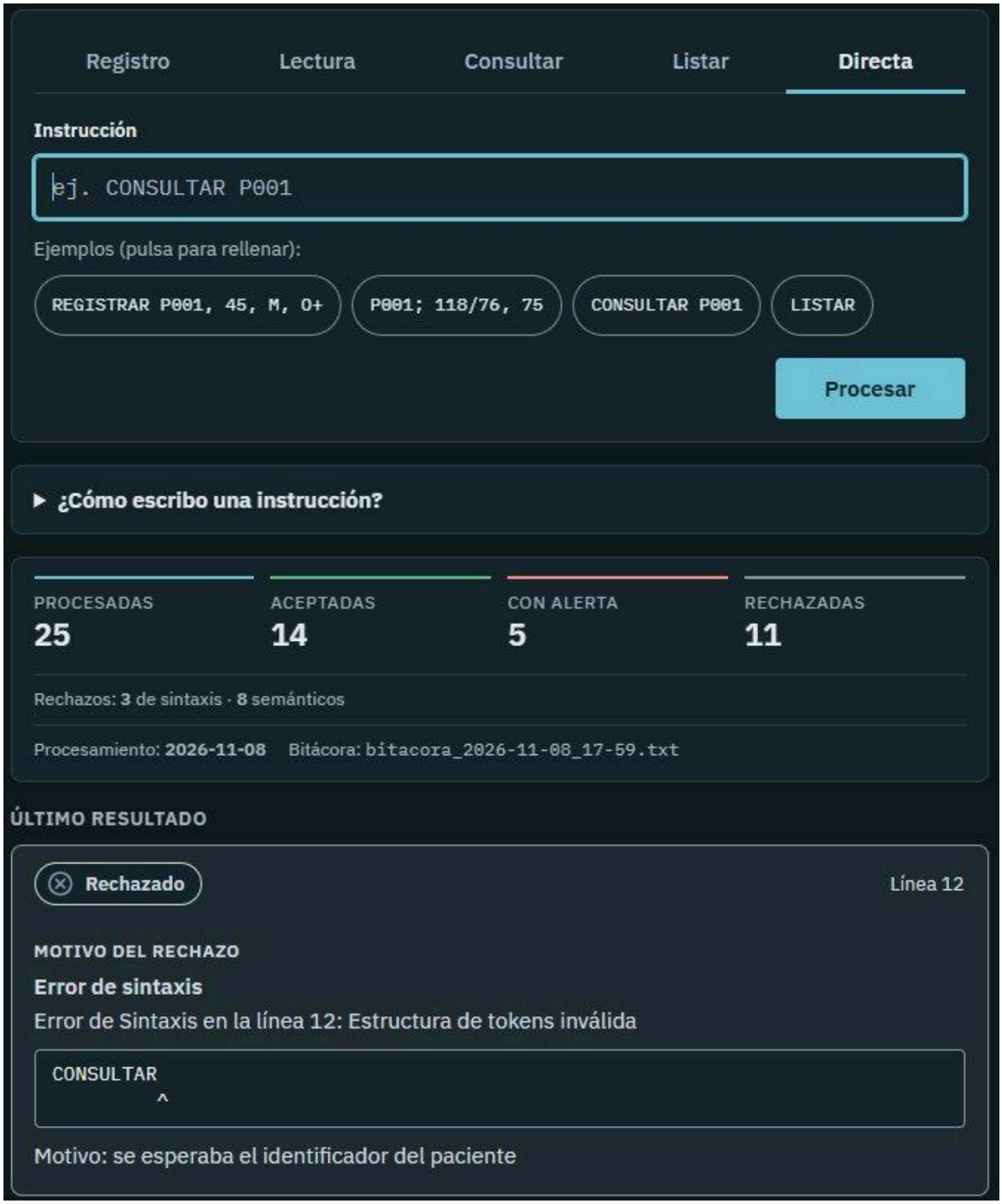


Figura 9.17. Error de sintaxis: CONSULTAR sin ID, con el marcador al final (captura 25).

9.6 Consultar un paciente

1. Abra la pestaña **Consultar**.
2. Escriba o elija el **ID del paciente**.
3. Pulse **Procesar**.

La ficha muestra la edad, el sexo, el tipo de sangre y la última revisión con su clasificación y sus etiquetas clínicas. Cuando los datos se guardan en SQL Server, la ficha incluye el **Historial de revisiones** con las 10 más recientes, la primera marcada como «Más reciente» (sección 7, figuras 7.7 y 7.13). Consultar no modifica ningún dato, así que la instrucción aparece como **Aceptado** aunque el paciente tenga una alerta en su última revisión.

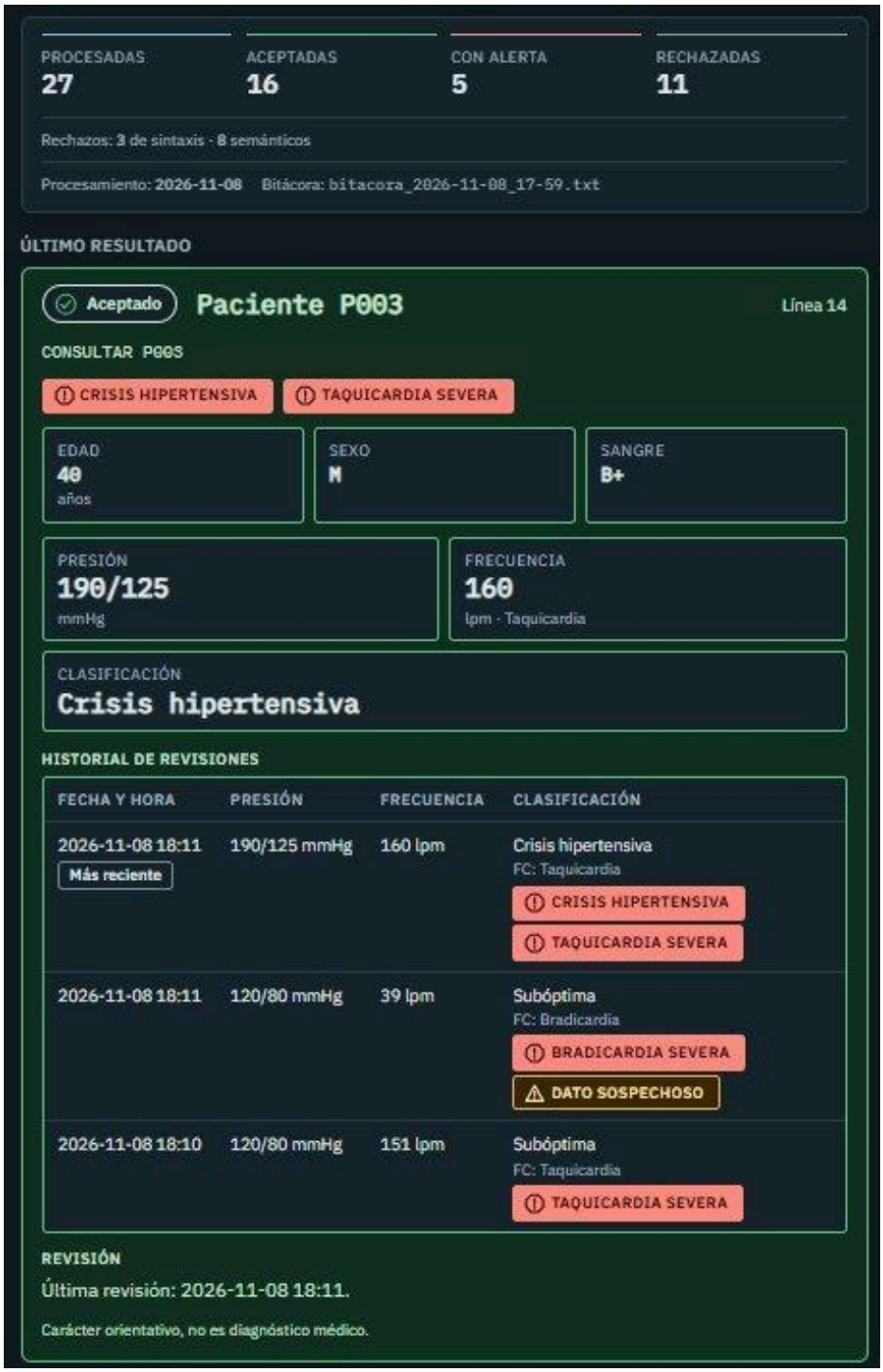


Figura 9.18. Ficha con etiquetas clínicas e historial de 3 revisiones (captura 27).

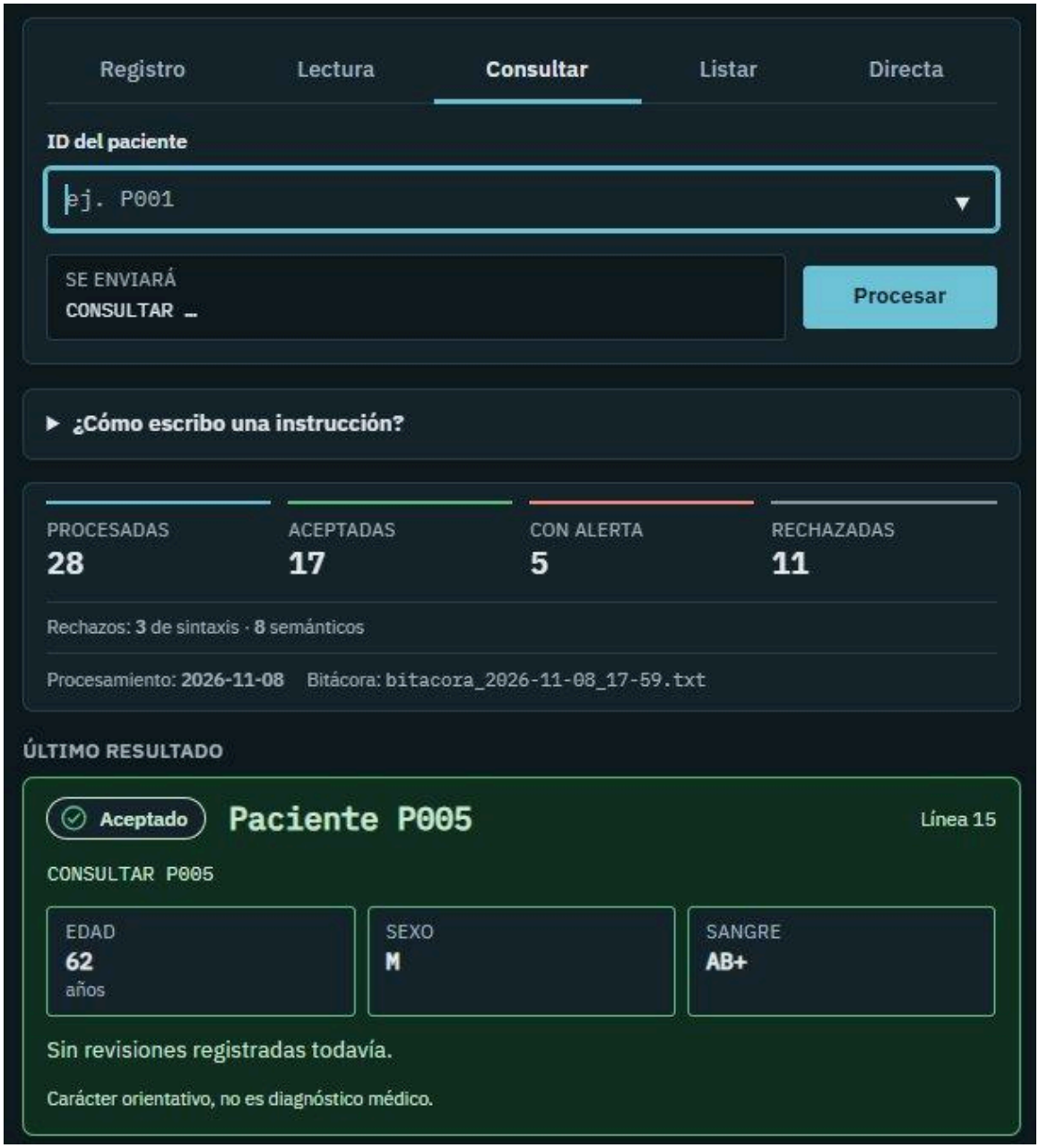


Figura 9.19. Paciente sin revisiones registradas (captura 28).

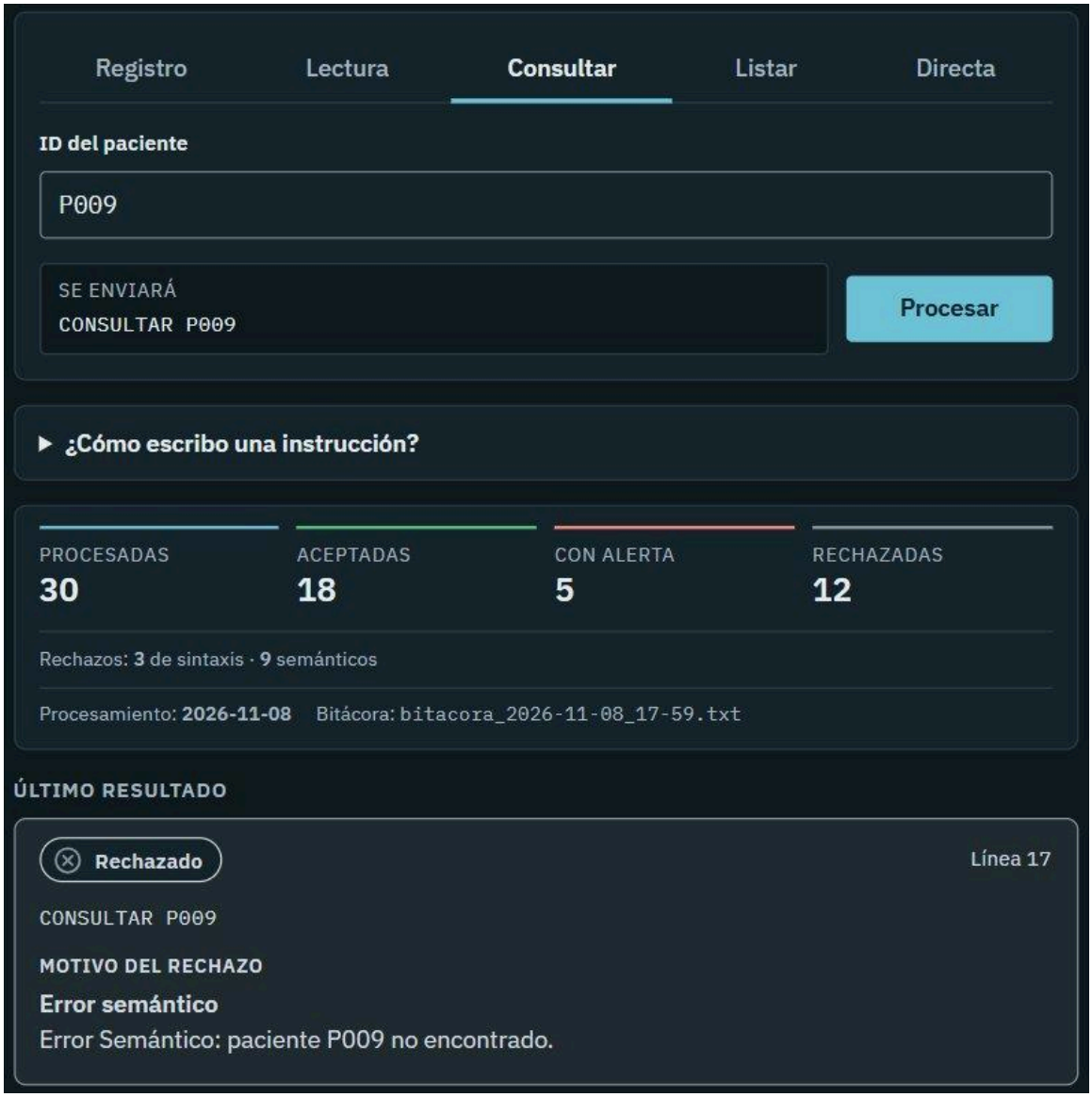


Figura 9.20. Rechazo al consultar un paciente que no existe (captura 30).

9.7 Listar los pacientes

1. Abra la pestaña **Listar**. No necesita datos.
2. Pulse **Procesar**.

Aparece una tabla con todos los pacientes registrados, su última revisión con fecha y hora, y su clasificación con etiquetas (sección 7, figura 7.8). Si todavía no hay pacientes, el resultado es **Aceptado** con el mensaje «No hay pacientes registrados» y un botón para registrar uno.

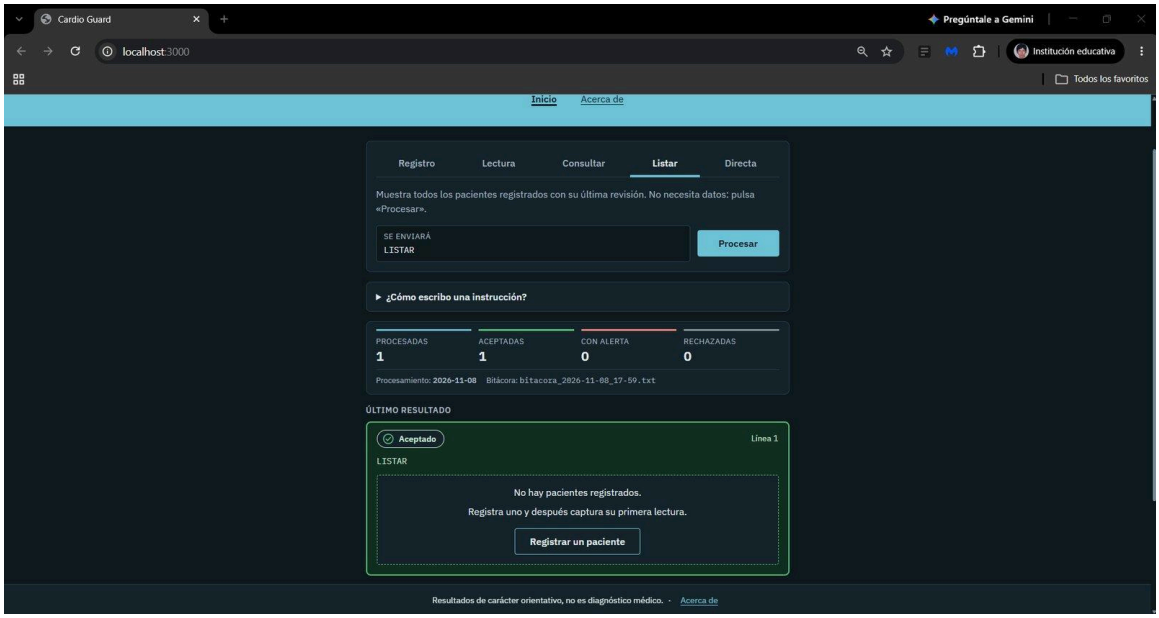


Figura 9.21. Listado sin pacientes registrados (captura 03).

9.8 Revisar la bitácora de la sesión

La **Bitácora de la sesión** está debajo del último resultado. Muestra una tarjeta por instrucción enviada desde que se abrió la página, la más reciente arriba.

1. Pulse un filtro: **Todas, Aceptadas, Alertas, Advertencias o Rechazadas**.
2. La línea «Mostrando N de M en esta sesión» indica cuántas tarjetas cumplen el filtro.
3. Pulse **Detalles** en una tarjeta para ver las alertas, el dato sospechoso o la comparación.

En los detalles, la comparación puede aparecer como `SIN_CAMBIO`, con guion bajo: es el texto tal como se guarda en el archivo de auditoría.

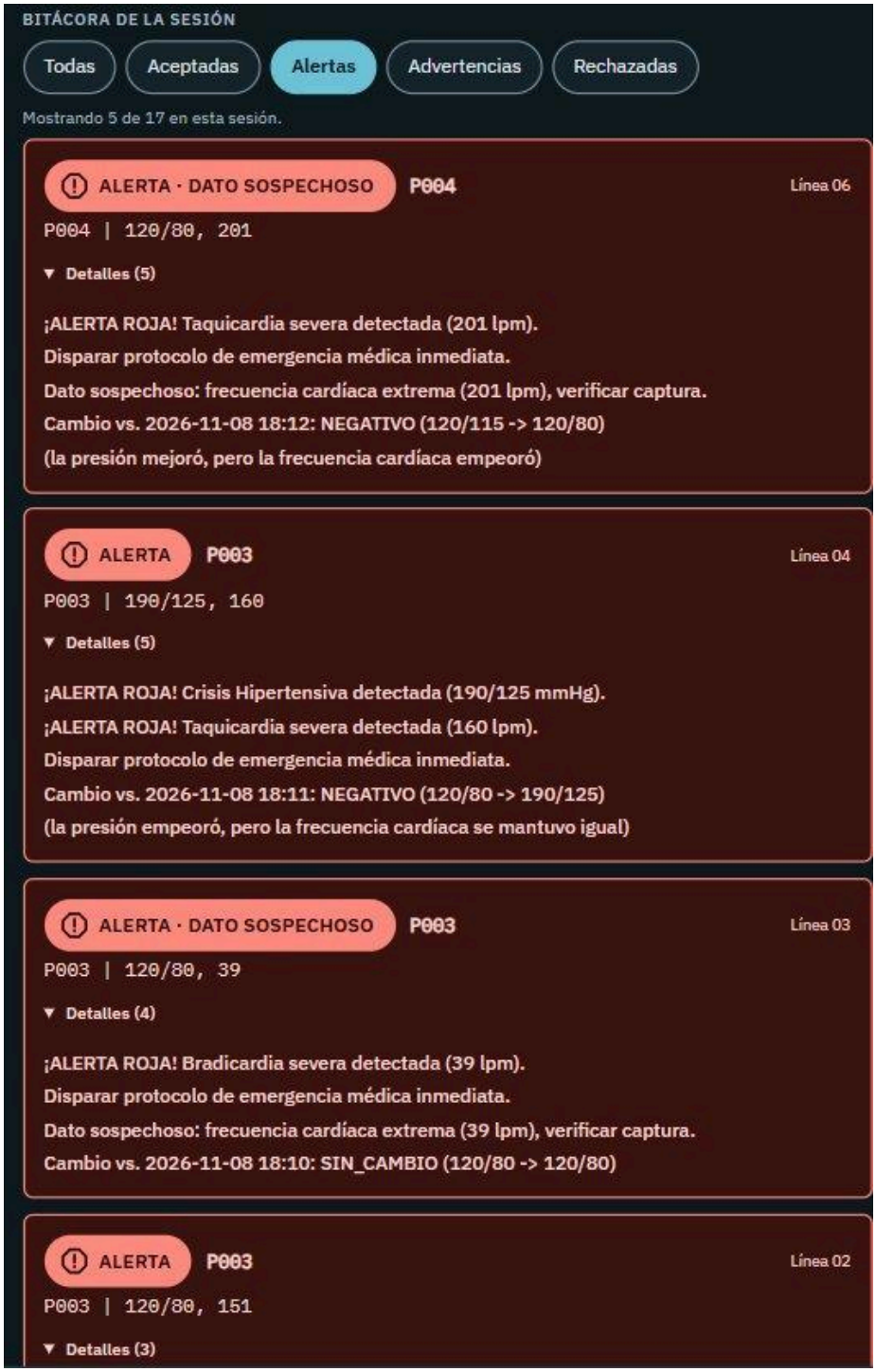


Figura 9.22. Bitácora filtrada por Alertas (captura 32).



Figura 9.23. Bitácora filtrada por Advertencias (captura 33).



Figura 9.24. Bitácora filtrada por Rechazadas, con el marcador ^ en los errores de sintaxis (captura 34).

9.9 Ayuda y uso en teléfono

- Pulse **¿Cómo escribo una instrucción?** para ver el formato de cada instrucción, cómo leer un error y el significado de cada estado (sección 7, figuras 7.10 y 7.11).
- Pulse **Acerca de** en la cabecera para ver la información del proyecto (figura 7.12).

- La página funciona igual en un teléfono; el pie con la leyenda «Resultados de carácter orientativo» se queda fijo y puede tapar el borde inferior (figuras 7.14 a 7.16).

9.10 Mensajes del sistema y solución de problemas

Situación	Qué significa	Qué hacer
«No se pudo acceder a la base de datos. Intenta de nuevo en un momento.»	El servidor no pudo conectarse a SQL Server. La instrucción se conserva en el campo.	Esperar unos segundos y pulsar **Procesar** otra vez. Si persiste, avisar al administrador.
La ficha dice «Solo se conserva la última revisión» y no muestra historial	El sistema está guardando los datos en archivo CSV en lugar de SQL Server.	Es normal en ese modo; el historial solo existe con SQL Server.
La página no abre	El servidor no está encendido.	Avisar al administrador.
Las líneas volvieron a empezar en 1	Se recargó la página.	Es normal; los contadores y el archivo de bitácora conservan todo.
Varias revisiones muestran la misma hora	Se enviaron en el mismo minuto.	Es normal; el orden lo da el orden de registro.



Figura 9.25. Aviso cuando la base de datos no está disponible; la instrucción se conserva (captura B1).

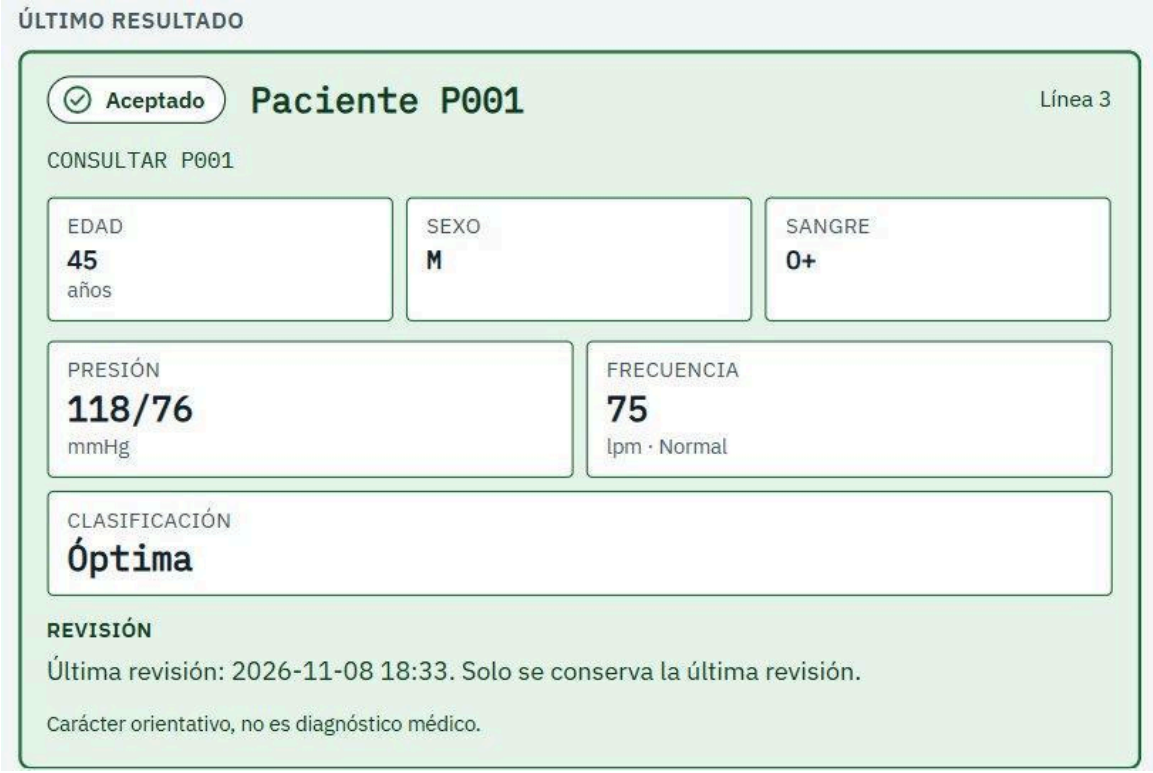


Figura 9.26. Ficha en modo CSV: solo se conserva la última revisión (captura C1).

Manual de administrador

Este manual explica cómo instalar, configurar, arrancar y mantener Cardio Guard. Está dirigido a quien administra el equipo donde corre el servidor; el uso diario de la interfaz está en el Manual de usuario (sección 9).

Cardio Guard puede guardar los pacientes de dos formas: en un archivo **CSV** (modo por defecto, sin instalar nada extra) o en **SQL Server** (necesario para el historial de revisiones). La bitácora de auditoría siempre se guarda en archivos de texto, en cualquiera de los dos modos.

A.1 Requisitos

Software	Para qué se usa	¿Obligatorio?
Node.js (probado con la versión 22) y npm	Ejecutar el servidor y el intérprete	Sí
Navegador web (Chrome, Edge o Firefox)	Usar la interfaz en http://localhost:3000	Sí
Git	Descargar el proyecto desde el repositorio	Recomendado
SQL Server Express (instancia `SQLEXPRESS`, TCP/IP en el puerto 1433, autenticación mixta)	Guardar pacientes e historial en base de datos	Solo en modo SQL
SQL Server Management Studio (SSMS)	Crear la base, el usuario y revisar los datos	Solo en modo SQL

A.2 Descargar e instalar el proyecto

1. Descargue el proyecto desde el repositorio: `git clone https://github.com/23212707-omar/cardio-guard`.
2. Entre a la carpeta del proyecto: `cd cardio-guard`.
3. Instale las dependencias con `npm install`. Se instalan tres paquetes: **express** (servidor web), **mssql** (conexión a SQL Server) y **dotenv** (lectura del archivo `.env`).

Con esto el sistema ya funciona en modo CSV. Para usar SQL Server, siga también los pasos A.3 y A.4.

A.3 Configurar el archivo .env

La configuración vive en un archivo `.env` en la carpeta del proyecto. Copie `.env.example` con el nombre `.env` y ajuste los valores. El archivo `.env` contiene la contraseña de la base de datos y **no se sube al repositorio**.

Variable	Valor por defecto	Qué indica
`REPOSITORIO`	`csv`	Dónde se guardan los pacientes: `csv` o `sql`.

Variable	Valor por defecto	Qué indica
`PUERTO`	`3000`	Puerto del servidor web.
`DB_SERVER`	`localhost`	Servidor de SQL Server.
`DB_PORT`	`1433`	Puerto TCP de SQL Server.
`DB_NAME`	`cardio_guard`	Nombre de la base de datos.
`DB_USER`	`cardio_guard_app`	Usuario de la aplicación.
`DB_PASSWORD`	(sin valor real)	Contraseña del usuario de la aplicación.
`DB_ENCRYPT`	`false`	Cifrado de la conexión.
`DB_TRUST_SERVER_CERTIFICATE`	`true`	Aceptar el certificado del servidor local.
`DB_TEST_USER`, `DB_TEST_PASSWORD`	(vacíos)	Solo para la prueba de SQL Server (A.8).

A.4 Preparar SQL Server (solo modo SQL)

1. En **SQL Server Configuration Manager**, habilite el protocolo **TCP/IP** de la instancia `SQLEXPRESS` en el puerto **1433** y confirme que la autenticación mixta (Windows y SQL Server) esté activa.
2. Abra SSMS con un usuario administrador y ejecute `sql/crear_base.sql`. Crea la base `cardio_guard` con las tablas `pacientes` y `revisiones`.
3. Abra `sql/crear_usuario.sql`, cambie `CAMBIAR_ESTA_CONTRASENA` por una contraseña real y ejecútelo. Crea el usuario `cardio_guard_app`.
4. En el archivo `.env`, escriba `REPOSITORIO=sql` y la misma contraseña en `DB_PASSWORD`.

El usuario `cardio_guard_app` solo tiene los roles `db_datareader` y `db_datawriter`: puede leer y escribir datos, pero no crear ni borrar tablas o bases. Los dos scripts se pueden ejecutar varias veces sin errores: solo crean lo que falta.

Tabla	Qué guarda	Columnas
`pacientes`	Un renglón por paciente con su última revisión	`orden`, `id` (llave primaria, hasta 20 caracteres), `edad`, `sexo`, `sangre`, `fecha_revision`, `sistolica`, `diastolica`, `frecuencia` (las cuatro últimas en `NULL` si no hay revisiones)
`revisiones`	Un renglón por cada lectura **aceptada** (historial)	`id_revision`, `paciente_id` (llave foránea a `pacientes`), `fecha_revision`, `sistolica`, `diastolica`, `frecuencia`

Las tablas guardan solo valores numéricos; la clasificación, las alertas y las etiquetas se calculan cada vez que se muestran. Si ya existía una base anterior sin historial, ejecute `sql/crear_base.sql` y después

`sql/migrar_historial.sql`, que copia la última revisión de cada paciente a la tabla `revisiones` sin duplicar datos.

A.5 Arrancar y detener el servidor

1. En una terminal dentro de la carpeta del proyecto, ejecute `npm start`.
2. La consola muestra la dirección del servidor, el repositorio activo (CSV o SQL) y el nombre del archivo de bitácora de esa ejecución. La contraseña nunca se imprime.
3. Indique a los usuarios que entren a **http://localhost:3000**.
4. Para detener el servidor, presione **Ctrl + C** en la misma terminal.

Para usar otro puerto, defina `PUERTO` en el archivo `.env` o al arrancar (`PUERTO=4000 npm start`; en PowerShell, `$env:PUERTO = 4000` y luego `npm start`).



Figura A.1. Consola al arrancar el servidor en modo SQL Server (captura 01).

Si `REPOSITORIO=sql` y el servidor no logra conectarse a la base al arrancar, muestra qué falló y qué variables revisar, y se detiene. No usa el CSV como respaldo, para no dividir los datos en dos lugares. Si SQL Server se detiene mientras el servidor ya está corriendo, los usuarios ven el aviso «No se pudo acceder a la base de datos» (sección 9, figura 9.25); al volver a iniciar el servicio, el servidor se recupera solo.

A.6 Archivos que genera el sistema

Todo se guarda en la carpeta `data/`, que no se sube al repositorio.

Archivo	Cuándo se crea	Contenido
`data/pacientes.csv`	Al arrancar en modo CSV, si no existe	Los pacientes y su última revisión. Cada lectura aceptada reemplaza a la anterior.
`data/bitacoras/bitacora_FECHA_HORA.txt`	Una vez por cada arranque del servidor	Una entrada por instrucción procesada, con su línea, estado, alertas, comparación y, en los errores, el marcador `^`.

Las bitácoras nunca se sobrescriben: cada arranque crea un archivo nuevo con la fecha y hora en el nombre. Se pueden abrir con el Bloc de notas. No conviene abrir `pacientes.csv` en Excel y guardarlo, porque Excel cambia el formato de las fechas; para revisarlo, ábralo con el Bloc de notas.

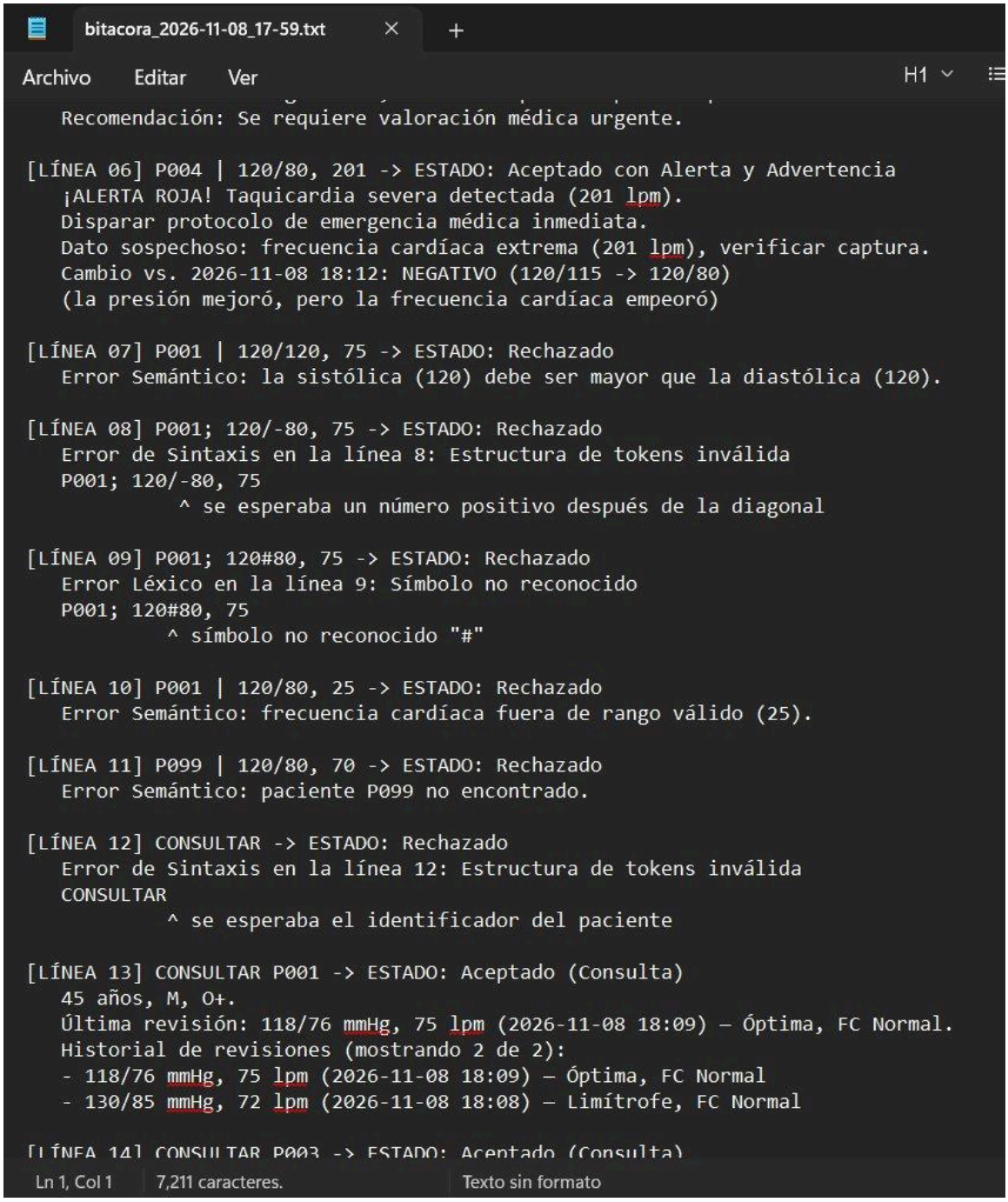
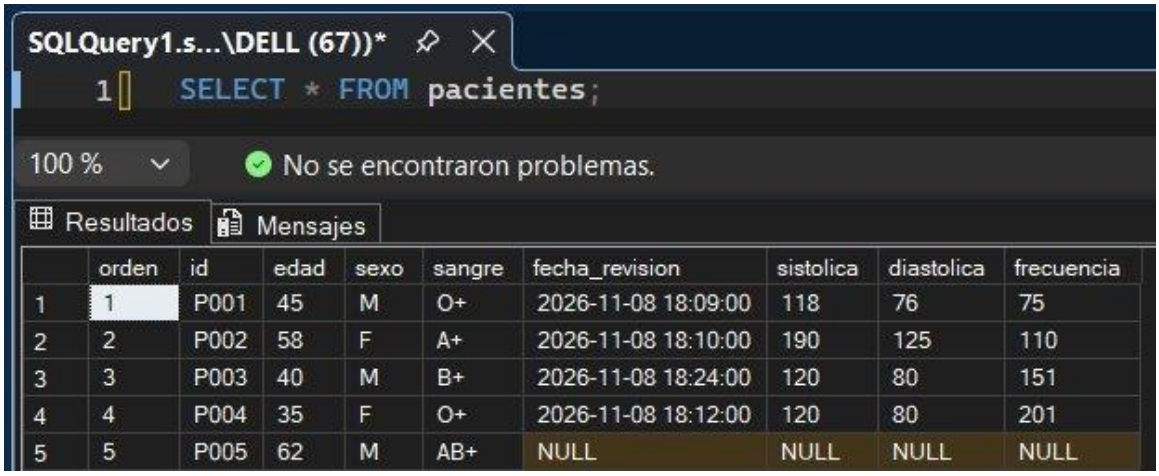


Figura A.2. Archivo de bitácora de una ejecución, abierto en el Bloc de notas (captura 42).

A.7 Revisar los datos en SQL Server

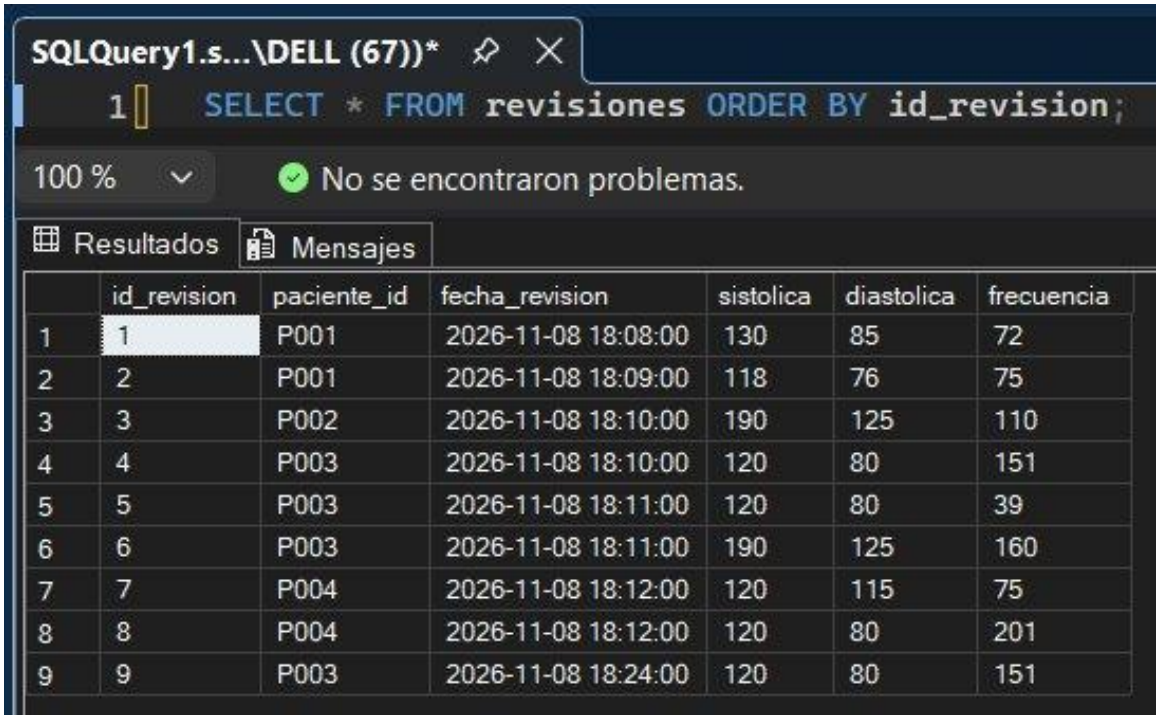
En SSMS, con la base `cardio_guard` seleccionada:

- `SELECT * FROM pacientes;` muestra cada paciente con su última revisión. Un paciente sin lecturas tiene esas columnas en `NULL`.
- `SELECT * FROM revisiones ORDER BY id_revision;` muestra el historial completo. Solo aparecen las lecturas aceptadas: las rechazadas nunca se guardan.



	orden	id	edad	sexo	sangre	fecha_revision	sistolica	diastolica	frecuencia
1	1	P001	45	M	O+	2026-11-08 18:09:00	118	76	75
2	2	P002	58	F	A+	2026-11-08 18:10:00	190	125	110
3	3	P003	40	M	B+	2026-11-08 18:24:00	120	80	151
4	4	P004	35	F	O+	2026-11-08 18:12:00	120	80	201
5	5	P005	62	M	AB+	NULL	NULL	NULL	NULL

Figura A.3. Tabla pacientes en SSMS (captura 40).



	id_revision	paciente_id	fecha_revision	sistolica	diastolica	frecuencia
1	1	P001	2026-11-08 18:08:00	130	85	72
2	2	P001	2026-11-08 18:09:00	118	76	75
3	3	P002	2026-11-08 18:10:00	190	125	110
4	4	P003	2026-11-08 18:10:00	120	80	151
5	5	P003	2026-11-08 18:11:00	120	80	39
6	6	P003	2026-11-08 18:11:00	190	125	160
7	7	P004	2026-11-08 18:12:00	120	115	75
8	8	P004	2026-11-08 18:12:00	120	80	201
9	9	P003	2026-11-08 18:24:00	120	80	151

Figura A.4. Tabla revisiones en SSMS: una fila por cada lectura aceptada (captura 41).

A.8 Pruebas del sistema

Las pruebas son scripts que se ejecutan con Node desde la carpeta del proyecto y muestran el resultado en la consola. No necesitan el servidor encendido.

Comando	Qué prueba
`node tests/lexer.manual.js`	Análisis léxico: tokens y errores léxicos
`node tests/parser.manual.js`	Análisis sintáctico: gramática y errores de sintaxis
`node tests/afd.manual.js`	Autómata finito determinista: transiciones y estados
`node tests/semantics.manual.js`	Reglas semánticas, clasificación y comparación
`node tests/diagnostico.manual.js`	Marcador `^` de los errores
`node tests/storage.manual.js`	Archivo CSV y bitácora
`node tests/storage.sql.manual.js`	Repositorio de SQL Server (se omite si no hay SQL Server)
`node tests/interpreter.manual.js`	Prueba de integración completa del intérprete
`node tests/api.pacientes.manual.js`	API del servidor, en un puerto temporal

La prueba de SQL Server necesita `DB_TEST_USER` y `DB_TEST_PASSWORD` en el `.env`, con un usuario que pueda crear bases de datos (`cardio_guard_app` no puede). La prueba crea y borra su propia base `cardio_guard_test` y nunca toca `cardio_guard`.

A.9 Interfaz de programación (API)

La página web se comunica con el servidor por tres rutas. Están documentadas como referencia para mantenimiento.

Ruta	Qué hace
`POST /api/procesar`	Recibe `{ texto, numeroLinea }`, pasa la instrucción por el intérprete y responde `{ resultado, lineasBitacora, contadores }`.
`GET /api/estado`	Responde la fecha de procesamiento, el archivo de bitácora en uso y los contadores.
`GET /api/pacientes`	Responde la lista de IDs registrados, para la lista desplegable de la interfaz. No pasa por el intérprete ni cuenta en la bitácora.

A.10 Respaldo y solución de problemas

Para respaldar la información, copie la carpeta `data/` completa y, en modo SQL, haga un respaldo de la base `cardio_guard` desde SSMS (clic derecho en la base, **Tareas**, **Copia de seguridad**).

Problema	Causa probable	Solución
Al arrancar: «REPOSITORIO=… no es válido»	Valor distinto de `csv` o `sql` en el `.env`	Corregir `REPOSITORIO`.
Al arrancar: «faltan credenciales de la base de datos»	`DB_USER` o `DB_PASSWORD` vacíos	Completarlos en el `.env`.
Al arrancar: «No se pudo conectar a SQL Server»	Servicio detenido, TCP/IP deshabilitado, puerto o contraseña incorrectos	Iniciar el servicio, habilitar TCP/IP en el puerto 1433 y revisar `DB_SERVER`, `DB_PORT`, `DB_NAME`, `DB_USER` y `DB_PASSWORD`.
El puerto 3000 ya está en uso	Otro programa o un servidor anterior sigue abierto	Cerrar el otro servidor con Ctrl + C o usar otro `PUERTO`.
La ficha de un paciente no muestra historial	El sistema está en modo CSV	Es normal: el historial solo existe con `REPOSITORIO=sql`.
Faltan pacientes al cambiar de modo	CSV y SQL Server son almacenes separados	Los datos no se copian solos de uno a otro; usar siempre el mismo modo.

10. Conclusiones

Se cumplió el objetivo general: Cardio Guard analiza cada instrucción con las tres fases de un traductor, la guía con un autómata finito determinista y responde con un resultado claro, ya sea la aceptación con su clasificación o el rechazo con el tipo de error y el lugar donde ocurrió.

El proyecto mostró que los conceptos de la materia sirven fuera de un compilador tradicional. Las expresiones regulares del analizador léxico, la gramática LL(1) del sintáctico y la quíntupla del AFD se tradujeron casi directamente a código: una lista ordenada de patrones, una función por cada regla de la gramática y una tabla de transiciones. Separar las fases también permitió dar mensajes de error específicos: no es lo mismo un símbolo no reconocido, una coma que falta o una sistólica menor que la diastólica, y el sistema distingue los tres.

El AFD no fue solo un requisito formal. Al revisar que cada estado pudiera alcanzarse, el equipo descubrió que el estado de advertencia existía pero ninguna lectura llegaba a él, lo que llevó a definir las causas del dato sospechoso. Esto confirmó que un estado inalcanzable es un error de diseño aunque el programa no falle.

En la parte práctica, el equipo aprendió a fijar reglas clínicas con una fuente oficial (la norma mexicana en lugar de una guía extranjera), a señalar qué umbrales eligió el propio equipo, a cuidar detalles que fallan en silencio (como guardar la fecha en hora UTC en lugar de la hora local) y a probar contra un servidor de base de datos real, porque varios problemas de SQL Server solo aparecieron al conectarse de verdad. El trabajo con ramas y pull requests permitió avanzar en la interfaz y en el intérprete al mismo tiempo sin romper lo que ya funcionaba.

El sistema tiene límites claros: es una herramienta de apoyo y no sustituye la valoración de un médico, no maneja usuarios ni contraseñas, y el historial completo solo está disponible con SQL Server. Dentro de esos límites, cumple con lo que se propuso y deja una base que se puede ampliar.

11. Recomendaciones

Para quien use el sistema

- Usar el modo SQL Server cuando se necesite ver la evolución de un paciente: el modo CSV solo guarda la última revisión.
- No abrir y guardar el archivo `pacientes.csv` con Excel, porque puede cambiar su formato. Para revisarlo, abrirlo sin guardar o consultar con `LISTAR`.
- Atender siempre una alerta roja con el procedimiento clínico del hospital; la clasificación es orientativa.
- Si una lectura sale como dato sospechoso, volver a medir antes de tomar decisiones.

Para quien administre el sistema

- No subir el archivo `.env` al repositorio y cambiar la contraseña de ejemplo del usuario `cardio_guard_app`.
- Respaldar la base de datos y la carpeta de bitácoras con regularidad (ver A.10).
- Antes de una demostración con SQL Server, tener listo el plan B: cambiar a `REPOSITORIO=csv` en `.env` y reiniciar el servidor, que funciona sin configuración adicional.

Para versiones futuras

- Agregar inicio de sesión con usuarios y roles, para saber quién capturó cada lectura.
- Programar el modo de entrada por archivo o por consola, que quedó en el diseño pero no se implementó, para procesar varias instrucciones a la vez.
- Convertir las pruebas manuales (`tests/*.manual.js`) en pruebas automáticas que se ejecuten con cada cambio.
- Pedir a un médico que revise los umbrales que eligió el equipo (frecuencia severa menor de 40 o mayor de 150 lpm, y las causas de dato sospechoso).
- Capturar condiciones como diabetes o enfermedad renal, para aplicar las metas de presión distintas que la norma indica para esos pacientes.
- Si el sistema se usa en una red, servirlo con HTTPS.

12. Bibliografía

Aho, A. V., Lam, M. S., Sethi, R. y Ullman, J. D. (2008). *Compiladores: principios, técnicas y herramientas* (2.ª ed.). Pearson Educación.

Campos-Nonato, I. et al. (2025). Hipertensión arterial en adultos y brechas de atención a nivel nacional y estatal, Ensanut 2021-2024. *Salud Pública de México, 67*(6), 633-643. https://doi.org/10.21149/17102

Express. (s.f.). *Express: Node.js web application framework*. https://expressjs.com/

Hopcroft, J. E., Motwani, R. y Ullman, J. D. (2007). *Introducción a la teoría de autómatas, lenguajes y computación* (3.ª ed.). Pearson Educación.

Microsoft. (s.f.). *Documentación de SQL Server*. https://learn.microsoft.com/es-es/sql/sql-server/

Node.js. (s.f.). *Node.js documentation*. https://nodejs.org/docs/latest/api/

Secretaría de Salud. (2010). Norma Oficial Mexicana NOM-030-SSA2-2009, Para la prevención, detección, diagnóstico, tratamiento y control de la hipertensión arterial sistémica. *Diario Oficial de la Federación*, 31 de mayo de 2010.

Secretaría de Salud. (2017). Proyecto de Norma Oficial Mexicana PROY-NOM-030-SSA2-2017, Para la prevención, detección, diagnóstico, tratamiento y control de la hipertensión arterial sistémica. *Diario Oficial de la Federación*.

Whelton, P. K., Carey, R. M., Aronow, W. S. et al. (2018). 2017 ACC/AHA/AAPA/ABC/ACPM/AGS/APhA/ASH/ASPC/NMA/PCNA guideline for the prevention, detection, evaluation, and management of high blood pressure in adults. *Hypertension, 71*(6), e13-e115. https://doi.org/10.1161/HYP.0000000000000065

23212707-omar. (2026). *Cardio Guard* [Repositorio de código]. GitHub. https://github.com/23212707-omar/cardio-guard

13. Anexos

Anexo A. Repositorio y estructura del proyecto

El código fuente completo está en https://github.com/23212707-omar/cardio-guard. La versión documentada corresponde al commit `20cf6dd` de la rama `main`.

```
cardio-guard/
├── .env.example              plantilla de configuración
├── package.json              dependencias: express, mssql, dotenv
├── README.md
├── DESIGN.md                 sistema de diseño de la interfaz
├── docs/
│   └── Historial_del_Proyecto.md
├── public/                   interfaz web
│   ├── index.html
│   ├── about.html
│   ├── app.js
│   ├── style.css
│   └── fonts/
├── sql/
│   ├── crear_base.sql
│   ├── crear_usuario.sql
│   └── migrar_historial.sql
├── src/
│   ├── server.js             servidor Express y API
│   ├── interpreter.js        orquestación de las fases y del AFD
│   ├── diagnostico.js
│   ├── lexer/lexer.js
│   ├── parser/parser.js
│   ├── automaton/afd.js
│   ├── semantics/
│   │   ├── validarRegistro.js
│   │   ├── validarLectura.js
│   │   ├── validarConsulta.js
│   │   └── clasificacion.js
│   └── storage/
│       ├── seleccionarRepo.js
│       ├── pacientesRepo.js      repositorio CSV
│       ├── pacientesRepoSql.js   repositorio SQL Server
│       └── bitacoraRepo.js
└── tests/                    pruebas manuales por módulo
    ├── lexer.manual.js
    ├── parser.manual.js
    ├── afd.manual.js
    ├── semantics.manual.js
    ├── interpreter.manual.js
    ├── storage.manual.js
    ├── storage.sql.manual.js
    ├── api.pacientes.manual.js
    └── diagnostico.manual.js
```

Anexo B. Código fuente del analizador

Se incluyen completos los tres módulos que implementan el análisis léxico, el sintáctico y el autómata. El resto de los archivos está en el repositorio.

B.1 `src/lexer/lexer.js`

```
'use strict';

/**
 * lexer.js — Analizador léxico de Cardio Guard
 *
 * Convierte una línea de texto (una instrucción, ya sea REGISTRAR o una
 * lectura) en una lista de tokens. No valida gramática (eso es trabajo
 * del parser) ni coherencia clínica (eso es trabajo del motor semántico);
 * solo reconoce qué "forma" tiene cada pedazo del texto.
 *
 * Tokens que puede producir (ver sección 4.2 del documento de contexto):
 *   REGISTRAR    palabra reservada
 *   CONSULTAR    palabra reservada
 *   LISTAR       palabra reservada
 *   ID           letra P seguida de uno o más dígitos (ej. P001)
 *   NUM          uno o más dígitos (ej. 118)
 *   SANGRE       una o más letras seguidas inmediatamente de + o -
 *                (ej. O+, AB-). Es la única forma de reconocer un tipo
 *                de sangre por su patrón, sin depender de contexto.
 *   LETRA        una o más letras sin signo (ej. M, F). El parser decide,
 *                por su posición en la gramática, si esto corresponde al
 *                campo "sexo" de un REGISTRAR. El lexer no distingue
 *                SEXO de cualquier otra letra suelta: eso no es un hecho
 *                léxico, es un hecho sintáctico/semántico.
 *   SLASH        /
 *   COMA         ,
 *   PUNTOYCOMA   ;
 *   GUION        -  (token propio, necesario para poder detectar casos
 *                    inválidos como "120/-80")
 *   EOF          marca el final de la línea (no corresponde a texto)
 */

const TIPO = Object.freeze({
  REGISTRAR: 'REGISTRAR',
  CONSULTAR: 'CONSULTAR',
  LISTAR: 'LISTAR',
  ID: 'ID',
  NUM: 'NUM',
  SANGRE: 'SANGRE',
  LETRA: 'LETRA',
  SLASH: 'SLASH',
  COMA: 'COMA',
  PUNTOYCOMA: 'PUNTOYCOMA',
  GUION: 'GUION',
  EOF: 'EOF',
});

/** Error léxico: símbolo que no coincide con ningún token válido. */
```

```
class ErrorLexico extends Error {
  /**
   * @param {string} detalle - qué falló, en minúsculas (ej. 'símbolo no reconocido
"@"');
   *   es el texto que se muestra junto al marcador "^".
   * @param {number} posicion - índice del carácter con la falla dentro de la línea.
   */
  constructor(detalle, posicion) {
    super(detalle.charAt(0).toUpperCase() + detalle.slice(1));
    this.name = 'ErrorLexico';
    this.detalle = detalle;
    this.posicion = posicion;
  }
}

// El orden importa: las reglas más específicas van primero, para que una
// coincidencia más general (como LETRA) no "robe" un texto que en realidad
// pertenece a una regla más concreta (como REGISTRAR o SANGRE).
const REGLAS = [
  { tipo: TIPO.REGISTRAR, regex: /^REGISTRAR\b/ },
  { tipo: TIPO.CONSULTAR, regex: /^CONSULTAR\b/ },
  { tipo: TIPO.LISTAR, regex: /^LISTAR\b/ },
  { tipo: TIPO.ID, regex: /^P\d+/ },
  { tipo: TIPO.SANGRE, regex: /^[A-Za-z]+[+-]/ },
  { tipo: TIPO.NUM, regex: /^\d+/ },
  { tipo: TIPO.LETRA, regex: /^[A-Za-z]+/ },
  { tipo: TIPO.SLASH, regex: /^\// },
  { tipo: TIPO.COMA, regex: /^,/ },
  { tipo: TIPO.PUNTOYCOMA, regex: /^;/ },
  { tipo: TIPO.GUION, regex: /^-/ },
];

/**
 * Convierte una línea de texto en una lista de tokens.
 * @param {string} linea - la instrucción a analizar (sin salto de línea).
 * @returns {Array<{tipo: string, valor: string, posicion: number}>}
 * @throws {ErrorLexico} si encuentra un símbolo que no coincide con ningún token.
 */
function analizarLexico(linea) {
  const tokens = [];
  let resto = linea;
  let posicion = 0;

  while (resto.length > 0) {
    // Los espacios en blanco se ignoran y no generan token.
    const espacio = resto.match(/^\s+/);
    if (espacio) {
      posicion += espacio[0].length;
      resto = resto.slice(espacio[0].length);
      continue;
    }

    const coincidencia = REGLAS
      .map((regla) => ({ regla, m: resto.match(regla.regex) }))
      .find((r) => r.m !== null);
```

```
    if (!coincidencia) {
      throw new ErrorLexico(`símbolo no reconocido "${resto[0]}"`, posicion);
    }

    const { regla, m } = coincidencia;
    tokens.push({ tipo: regla.tipo, valor: m[0], posicion });
    posicion += m[0].length;
    resto = resto.slice(m[0].length);
  }

  tokens.push({ tipo: TIPO.EOF, valor: '', posicion });
  return tokens;
}

module.exports = { analizarLexico, ErrorLexico, TIPO };
```

B.2 src/parser/parser.js

```
'use strict';

const { TIPO } = require('../lexer/lexer');

/**
 * parser.js — Analizador sintáctico de Cardio Guard
 *
 * Gramática que implementa (ver sección 4.3 del documento de contexto):
 *   S   -> REG | LEC | CON | LIS
 *   REG -> registrar id , num , sexo , sangre
 *   LEC -> id ; P , F
 *   P   -> num / num
 *   F   -> num
 *   CON -> consultar id
 *   LIS -> listar
 *
 * Entrega un árbol ya "aplanado" a un objeto plano (no una estructura de
 * nodos anidados), porque es lo que el motor semántico necesita consumir:
 *
 *   REGISTRAR P001, 45, M, O+
 *     -> { tipo: 'REG', id: 'P001', edad: 45, sexo: 'M', sangre: 'O+' }
 *
 *   P001; 118/76, 75
 *     -> { tipo: 'LEC', id: 'P001', sistolica: 118, diastolica: 76, frecuencia: 75 }
 *
 *   CONSULTAR P001  -> { tipo: 'CON', id: 'P001' }
 *   LISTAR          -> { tipo: 'LIS' }
 *
 * El parser ya convierte los números de texto a tipo numérico; eso es un
 * paso mecánico, no una regla semántica.
 *
 * El parser NO sabe en qué línea del archivo está (solo ve una instrucción
 * a la vez), así que sus errores solo cargan la posición dentro de la
 * línea. Es responsabilidad de interpreter.js anteponer "línea X" al
 * mensaje final.
 */
```

```
/** Error de sintaxis: la secuencia de tokens no sigue la gramática. */
class ErrorSintactico extends Error {
  /**
   * @param {string} detalle - qué se esperaba, en minúsculas (ej. 'se esperaba
","');
   *   es el texto que se muestra junto al marcador "^".
   * @param {number} posicion - índice del carácter con la falla dentro de la línea.
   */
  constructor(detalle, posicion) {
    super(`Estructura de tokens inválida (${detalle})`);
    this.name = 'ErrorSintactico';
    this.detalle = detalle;
    this.posicion = posicion;
  }
}

/**
 * Convierte una lista de tokens (salida de analizarLexico) en el árbol
 * de la instrucción.
 * @param {Array<{tipo: string, valor: string, posicion: number}>} tokens
 * @returns {{tipo: 'REG', id: string, edad: number, sexo: string, sangre: string}
 *          | {tipo: 'LEC', id: string, sistolica: number, diastolica: number,
frecuencia: number}
 *          | {tipo: 'CON', id: string}
 *          | {tipo: 'LIS'}}
 * @throws {ErrorSintactico}
 */
function analizarSintactico(tokens) {
  let i = 0;

  const actual = () => tokens[i];

  const avanzar = () => {
    const t = tokens[i];
    i += 1;
    return t;
  };

  /** Exige que el token actual sea del tipo dado; si no, lanza ErrorSintactico. */
  const esperar = (tipoEsperado, mensaje) => {
    const t = actual();
    if (t.tipo !== tipoEsperado) {
      throw new ErrorSintactico(mensaje(t), t.posicion);
    }
    return avanzar();
  };

  // P -> num / num
  function parsearP() {
    const numIzq = esperar(
      TIPO.NUM,
      (t) => 'se esperaba la presión sistólica'
    ).valor;

    esperar(
```

```
    TIPO.SLASH,
    (t) => 'se esperaba "/"'
  );

  // Caso específico documentado: un guion justo después de la diagonal,
  // como en "120/-80". Se reporta con el mensaje exacto ya definido,
  // en vez del mensaje genérico de "se esperaba un número".
  if (actual().tipo === TIPO.GUION) {
    throw new ErrorSintactico(
      'se esperaba un número positivo después de la diagonal',
      actual().posicion
    );
  }

  const numDer = esperar(
    TIPO.NUM,
    (t) => 'se esperaba la presión diastólica'
  ).valor;

  return { sistolica: Number(numIzq), diastolica: Number(numDer) };
}

// F -> num
function parsearF() {
  const num = esperar(
    TIPO.NUM,
    (t) => 'se esperaba la frecuencia cardíaca'
  ).valor;
  return Number(num);
}

/** Exige que no quede nada más que EOF después de la instrucción. */
function exigirFinDeLinea() {
  esperar(
    TIPO.EOF,
    (t) => `sobra texto después de la instrucción: "${t.valor}"`
  );
}

// REG -> registrar id , num , sexo , sangre
function parsearREG() {
  esperar(TIPO.REGISTRAR, () => 'se esperaba REGISTRAR');
  const id = esperar(
    TIPO.ID,
    () => 'se esperaba el identificador del paciente'
  ).valor;
  esperar(TIPO.COMA, () => 'se esperaba ","');
  const edad = esperar(
    TIPO.NUM,
    () => 'se esperaba la edad'
  ).valor;
  esperar(TIPO.COMA, () => 'se esperaba ","');
  const sexo = esperar(
    TIPO.LETRA,
    () => 'se esperaba el sexo'
```

```
  ).valor;
  esperar(TIPO.COMA, () => 'se esperaba ","');
  const sangre = esperar(
    TIPO.SANGRE,
    () => 'se esperaba el tipo de sangre'
  ).valor;
  exigirFinDeLinea();

  return { tipo: 'REG', id, edad: Number(edad), sexo, sangre };
}

// LEC -> id ; P , F
function parsearLEC() {
  const id = esperar(
    TIPO.ID,
    () => 'se esperaba el identificador del paciente'
  ).valor;
  esperar(TIPO.PUNTOYCOMA, () => 'se esperaba ";"');
  const presion = parsearP();
  esperar(TIPO.COMA, () => 'se esperaba ","');
  const frecuencia = parsearF();
  exigirFinDeLinea();

  return {
    tipo: 'LEC',
    id,
    sistolica: presion.sistolica,
    diastolica: presion.diastolica,
    frecuencia,
  };
}

// CON -> consultar id
function parsearCON() {
  esperar(TIPO.CONSULTAR, () => 'se esperaba CONSULTAR');
  const id = esperar(
    TIPO.ID,
    () => 'se esperaba el identificador del paciente'
  ).valor;
  exigirFinDeLinea();
  return { tipo: 'CON', id };
}

// LIS -> listar
function parsearLIS() {
  esperar(TIPO.LISTAR, () => 'se esperaba LISTAR');
  exigirFinDeLinea();
  return { tipo: 'LIS' };
}

// S -> REG | LEC | CON | LIS   (se decide con el primer token, sin retroceder)
const primero = actual();
if (primero.tipo === TIPO.REGISTRAR) {
  return parsearREG();
}
```

```
  if (primero.tipo === TIPO.CONSULTAR) {
    return parsearCON();
  }
  if (primero.tipo === TIPO.LISTAR) {
    return parsearLIS();
  }
  if (primero.tipo === TIPO.ID) {
    return parsearLEC();
  }
  throw new ErrorSintactico(
    'se esperaba REGISTRAR, CONSULTAR, LISTAR o el identificador de un paciente',
    primero.posicion
  );
}

module.exports = { analizarSintactico, ErrorSintactico };
```

B.3 src/automaton/afd.js

```
'use strict';

/**
 * afd.js — Autómata Finito Determinista de Cardio Guard
 *
 * Es una máquina de estados "pura": no llama al lexer, al parser ni a las
 * reglas semánticas. Solo recibe EVENTOS (el resultado de cada etapa) y
 * decide a qué ESTADO pasar, según la tabla de transiciones. Quien orquesta
 * el proceso completo (llamar al lexer, al parser, etc. y alimentar al AFD
 * con los eventos correspondientes) es interpreter.js.
 *
 * Definición formal (5-tupla, sección 7/13 del documento de contexto):
 *   Q  -> el objeto ESTADO, de abajo
 *   Σ  -> el objeto EVENTO, de abajo
 *   q0 -> ESTADO.INICIO
 *   δ  -> la tabla TRANSICIONES
 *   F  -> ESTADOS_DE_ACEPTACION (ACEPTADO y sus variantes con alerta/advertencia)
 *
 * RECHAZADO es un estado final, pero queda fuera de F a propósito: la
 * documentación distingue "aceptado" de "rechazado" como dos desenlaces
 * distintos, no F vs. "lo que no es F".
 */

const ESTADO = Object.freeze({
  INICIO: 'INICIO',
  VALIDANDO_SEMANTICA_REGISTRO: 'VALIDANDO_SEMANTICA_REGISTRO',
  BUSCANDO_PACIENTE: 'BUSCANDO_PACIENTE',
  VALIDANDO_SEMANTICA_LECTURA: 'VALIDANDO_SEMANTICA_LECTURA',
  CLASIFICANDO: 'CLASIFICANDO',
  BUSCANDO_PACIENTE_CONSULTA: 'BUSCANDO_PACIENTE_CONSULTA',
  LISTANDO: 'LISTANDO',
  ACEPTADO: 'ACEPTADO',
  ACEPTADO_CON_ALERTA: 'ACEPTADO_CON_ALERTA',
  ACEPTADO_CON_ADVERTENCIA: 'ACEPTADO_CON_ADVERTENCIA',
  ACEPTADO_CON_ALERTA_Y_ADVERTENCIA: 'ACEPTADO_CON_ALERTA_Y_ADVERTENCIA',
  RECHAZADO: 'RECHAZADO',
});
```

```
const EVENTO = Object.freeze({
  LEXICO_SINTACTICO_ERROR: 'LEXICO_SINTACTICO_ERROR',
  LEXICO_SINTACTICO_OK_REG: 'LEXICO_SINTACTICO_OK_REG',
  LEXICO_SINTACTICO_OK_LEC: 'LEXICO_SINTACTICO_OK_LEC',
  LEXICO_SINTACTICO_OK_CON: 'LEXICO_SINTACTICO_OK_CON',
  LEXICO_SINTACTICO_OK_LIS: 'LEXICO_SINTACTICO_OK_LIS',
  CONSULTA_PACIENTE_NO_ENCONTRADO: 'CONSULTA_PACIENTE_NO_ENCONTRADO',
  CONSULTA_PACIENTE_ENCONTRADO: 'CONSULTA_PACIENTE_ENCONTRADO',
  LISTADO_OK: 'LISTADO_OK',
  SEMANTICA_REGISTRO_ERROR: 'SEMANTICA_REGISTRO_ERROR',
  SEMANTICA_REGISTRO_OK: 'SEMANTICA_REGISTRO_OK',
  PACIENTE_NO_ENCONTRADO: 'PACIENTE_NO_ENCONTRADO',
  PACIENTE_ENCONTRADO: 'PACIENTE_ENCONTRADO',
  SEMANTICA_LECTURA_ERROR: 'SEMANTICA_LECTURA_ERROR',
  SEMANTICA_LECTURA_OK: 'SEMANTICA_LECTURA_OK',
  CLASIFICACION_NORMAL: 'CLASIFICACION_NORMAL',
  CLASIFICACION_ALERTA: 'CLASIFICACION_ALERTA',
  CLASIFICACION_ADVERTENCIA: 'CLASIFICACION_ADVERTENCIA',
  CLASIFICACION_ALERTA_Y_ADVERTENCIA: 'CLASIFICACION_ALERTA_Y_ADVERTENCIA',
});

const ESTADOS_DE_ACEPTACION = new Set([
  ESTADO.ACEPTADO,
  ESTADO.ACEPTADO_CON_ALERTA,
  ESTADO.ACEPTADO_CON_ADVERTENCIA,
  ESTADO.ACEPTADO_CON_ALERTA_Y_ADVERTENCIA,
]);

const ESTADOS_FINALES = new Set([...ESTADOS_DE_ACEPTACION, ESTADO.RECHAZADO]);

// δ: función de transición, como tabla "estado:evento" -> estado siguiente.
const TRANSICIONES = Object.freeze({
  [`${ESTADO.INICIO}:${EVENTO.LEXICO_SINTACTICO_ERROR}`]: ESTADO.RECHAZADO,
  [`${ESTADO.INICIO}:${EVENTO.LEXICO_SINTACTICO_OK_REG}`]:
ESTADO.VALIDANDO_SEMANTICA_REGISTRO,
  [`${ESTADO.INICIO}:${EVENTO.LEXICO_SINTACTICO_OK_LEC}`]: ESTADO.BUSCANDO_PACIENTE,
  [`${ESTADO.INICIO}:${EVENTO.LEXICO_SINTACTICO_OK_CON}`]:
ESTADO.BUSCANDO_PACIENTE_CONSULTA,
  [`${ESTADO.INICIO}:${EVENTO.LEXICO_SINTACTICO_OK_LIS}`]: ESTADO.LISTANDO,

  [`${ESTADO.BUSCANDO_PACIENTE_CONSULTA}:${EVENTO.CONSULTA_PACIENTE_NO_ENCONTRADO}`]:
ESTADO.RECHAZADO,
  [`${ESTADO.BUSCANDO_PACIENTE_CONSULTA}:${EVENTO.CONSULTA_PACIENTE_ENCONTRADO}`]:
ESTADO.ACEPTADO,

  [`${ESTADO.LISTANDO}:${EVENTO.LISTADO_OK}`]: ESTADO.ACEPTADO,

  [`${ESTADO.VALIDANDO_SEMANTICA_REGISTRO}:${EVENTO.SEMANTICA_REGISTRO_ERROR}`]:
ESTADO.RECHAZADO,
  [`${ESTADO.VALIDANDO_SEMANTICA_REGISTRO}:${EVENTO.SEMANTICA_REGISTRO_OK}`]:
ESTADO.ACEPTADO,

  [`${ESTADO.BUSCANDO_PACIENTE}:${EVENTO.PACIENTE_NO_ENCONTRADO}`]: ESTADO.RECHAZADO,
  [`${ESTADO.BUSCANDO_PACIENTE}:${EVENTO.PACIENTE_ENCONTRADO}`]:
```

```
ESTADO.VALIDANDO_SEMANTICA_LECTURA,

  [`${ESTADO.VALIDANDO_SEMANTICA_LECTURA}:${EVENTO.SEMANTICA_LECTURA_ERROR}`]:
ESTADO.RECHAZADO,
  [`${ESTADO.VALIDANDO_SEMANTICA_LECTURA}:${EVENTO.SEMANTICA_LECTURA_OK}`]:
ESTADO.CLASIFICANDO,

  [`${ESTADO.CLASIFICANDO}:${EVENTO.CLASIFICACION_NORMAL}`]: ESTADO.ACEPTADO,
  [`${ESTADO.CLASIFICANDO}:${EVENTO.CLASIFICACION_ALERTA}`]:
ESTADO.ACEPTADO_CON_ALERTA,
  [`${ESTADO.CLASIFICANDO}:${EVENTO.CLASIFICACION_ADVERTENCIA}`]:
ESTADO.ACEPTADO_CON_ADVERTENCIA,
  [`${ESTADO.CLASIFICANDO}:${EVENTO.CLASIFICACION_ALERTA_Y_ADVERTENCIA}`]:
ESTADO.ACEPTADO_CON_ALERTA_Y_ADVERTENCIA,
});

/** Error de programación: se intentó una transición que la tabla no contempla. */
class TransicionInvalidaError extends Error {
  constructor(estado, evento) {
    super(
      `Transición inválida: el evento "${evento}" no está definido para el estado
"${estado}". ` +
      'Esto es un error de interpreter.js, no un error de los datos del usuario.'
    );
    this.name = 'TransicionInvalidaError';
    this.estado = estado;
    this.evento = evento;
  }
}

class AFD {
  constructor() {
    this.estado = ESTADO.INICIO;
    this.historial = [ESTADO.INICIO];
  }

  /** Aplica un evento y mueve el autómata al siguiente estado. */
  transitar(evento) {
    const clave = `${this.estado}:${evento}`;
    const siguiente = TRANSICIONES[clave];
    if (siguiente === undefined) {
      throw new TransicionInvalidaError(this.estado, evento);
    }
    this.estado = siguiente;
    this.historial.push(siguiente);
    return this.estado;
  }

  estadoActual() {
    return this.estado;
  }

  /** ¿El autómata llegó a un estado del que ya no sale (aceptación o rechazo)? */
  esFinal() {
    return ESTADOS_FINALES.has(this.estado);
```

```
  }

  /** ¿El estado actual es alguno de los de aceptación (F)? */
  esAceptado() {
    return ESTADOS_DE_ACEPTACION.has(this.estado);
  }

  esRechazado() {
    return this.estado === ESTADO.RECHAZADO;
  }
}

/** Crea un AFD nuevo en el estado inicial (q0), uno por cada instrucción procesada.
*/
function crearAFD() {
  return new AFD();
}

module.exports = {
  ESTADO,
  EVENTO,
  TRANSICIONES,
  ESTADOS_DE_ACEPTACION,
  ESTADOS_FINALES,
  TransicionInvalidaError,
  AFD,
  crearAFD,
};
```

Anexo C. Scripts de la base de datos

Se ejecutan en SQL Server Management Studio con un usuario administrador, en este orden (ver A.4).

C.1 `sql/crear_base.sql`

```
-- Cardio Guard: crea la base y la tabla de pacientes (idempotente).
-- Ejecutar en SSMS con un usuario administrador (p. ej. conexión de Windows).
-- Se puede correr varias veces sin errores: solo crea lo que falta.

IF DB_ID(N'cardio_guard') IS NULL
    CREATE DATABASE cardio_guard;
GO

USE cardio_guard;
GO

IF OBJECT_ID(N'dbo.pacientes', N'U') IS NULL
BEGIN
    CREATE TABLE dbo.pacientes (
        orden          INT IDENTITY(1,1) NOT NULL,  -- orden de registro (listar()
ordena por aquí)
        id             VARCHAR(20)  NOT NULL,
        edad           INT          NOT NULL,
        sexo           CHAR(1)      NOT NULL,
        sangre         VARCHAR(3)   NOT NULL,
```

```
        fecha_revision DATETIME2(0) NULL,           -- hora local "YYYY-MM-DD HH:MM",
NULL sin revisión
        sistolica      INT          NULL,
        diastolica     INT          NULL,
        frecuencia     INT          NULL,
        CONSTRAINT PK_pacientes PRIMARY KEY (id),
        CONSTRAINT UQ_pacientes_orden UNIQUE (orden)
    );
END
GO

-- Historial: una fila por cada lectura ACEPTADA (solo valores crudos; las categorias
-- y alertas se calculan al mostrarlas). pacientes conserva solo la ultima revision.
IF OBJECT_ID(N'dbo.revisiones', N'U') IS NULL
BEGIN
    CREATE TABLE dbo.revisiones (
        id_revision    INT IDENTITY(1,1) NOT NULL,
        paciente_id    VARCHAR(20)  NOT NULL,       -- misma longitud que
pacientes.id
        fecha_revision DATETIME2(0) NOT NULL,       -- hora local "YYYY-MM-DD HH:MM"
        sistolica      INT          NOT NULL,
        diastolica     INT          NOT NULL,
        frecuencia     INT          NOT NULL,
        CONSTRAINT PK_revisiones PRIMARY KEY (id_revision),
        CONSTRAINT FK_revisiones_pacientes FOREIGN KEY (paciente_id) REFERENCES
dbo.pacientes (id)
    );
END
GO

IF NOT EXISTS (SELECT 1 FROM sys.indexes
               WHERE name = N'IX_revisiones_paciente' AND object_id =
OBJECT_ID(N'dbo.revisiones'))
    CREATE INDEX IX_revisiones_paciente ON dbo.revisiones (paciente_id, id_revision);
GO
```

C.2 `sql/crear_usuario.sql`

```
-- Cardio Guard: login y usuario de la aplicación, SOLO lectura/escritura de datos.
-- IMPORTANTE: cambia CAMBIAR_ESTA_CONTRASENA por una contraseña real ANTES de
-- ejecutar este script, y no la subas al repositorio.
-- Ejecutar después de crear_base.sql, con un usuario administrador, en SSMS.
-- (No es sa, ni db_owner: no puede crear/borrar tablas ni bases.)

USE master;
GO

IF SUSER_ID(N'cardio_guard_app') IS NULL
    CREATE LOGIN cardio_guard_app
        WITH PASSWORD = N'CAMBIAR_ESTA_CONTRASENA', CHECK_POLICY = ON;
GO

USE cardio_guard;
GO

IF USER_ID(N'cardio_guard_app') IS NULL
```

```
    CREATE USER cardio_guard_app FOR LOGIN cardio_guard_app;
GO

ALTER ROLE db_datareader ADD MEMBER cardio_guard_app;
ALTER ROLE db_datawriter ADD MEMBER cardio_guard_app;
GO
```

C.3 `sql/migrar_historial.sql`

```
-- Cardio Guard: copia a revisiones la ultima revision de cada paciente que aun no
-- tiene historial (para que los pacientes ya guardados no empiecen vacios).
-- Idempotente: se puede correr varias veces; no duplica filas.
-- Ejecutar DESPUES de crear_base.sql (que crea la tabla revisiones), con un
administrador.

USE cardio_guard;
GO

IF OBJECT_ID(N'dbo.revisiones', N'U') IS NULL
    THROW 50001, 'Falta la tabla revisiones: ejecuta primero sql/crear_base.sql.', 1;
GO

INSERT INTO dbo.revisiones (paciente_id, fecha_revision, sistolica, diastolica,
frecuencia)
SELECT p.id, p.fecha_revision, p.sistolica, p.diastolica, p.frecuencia
  FROM dbo.pacientes AS p
 WHERE p.fecha_revision IS NOT NULL
   AND NOT EXISTS (SELECT 1 FROM dbo.revisiones AS r WHERE r.paciente_id = p.id);
GO
```

Anexo D. Problemas encontrados durante el desarrollo

Problema	Qué pasaba	Solución
Fecha en UTC	Las revisiones guardadas por la tarde en Tijuana quedaban con la fecha del día siguiente.	Se usa la hora local de la computadora y cada revisión guarda fecha y hora.
Comparación contra sí misma	La lectura nueva se comparaba consigo misma, porque el dato se leía después de actualizar el repositorio.	Se guarda la revisión anterior en una variable antes de actualizar.
Advertencia inalcanzable	El estado existía en el AFD, pero ninguna lectura podía llegar a él.	Se definieron causas propias del dato sospechoso.
ID demasiado largo	Un ID de más de 20 caracteres provocaba un error interno de SQL Server.	Rechazo semántico con límite de 20 caracteres.
No conectaba con SQL Server	La autenticación de Windows de SSMS no sirve para Node.js.	Usuario propio `cardio_guard_app`, autenticación mixta y TCP/IP en el puerto 1433.

Problema	Qué pasaba	Solución
Falla de la base durante el uso	Con el servicio detenido, el servidor mostraba errores técnicos.	Respuesta 503 con un mensaje en español; el servidor se recupera al volver el servicio.
Parpadeo de fuentes	El título aparecía un instante con otra fuente al navegar.	Precarga de fuentes y fuentes de respaldo con medidas ajustadas.
Datos confusos en el CSV	Guardar el CSV desde Excel cambiaba su contenido.	No guardar el CSV desde Excel.